DOSSIER DE PROJET

Concepteur Développeur d'Applications

ASSIST'IA

Application web immobilière — projet fil rouge IPSSI

Diana MARGARIAN
IPSSI
Août 2026

Certification CDA · Session 2026

Fiche d'identité

Nom	MARGARIAN
Prénom	Diana
Formation	IPSSI — Concepteur Développeur d'Applications
Projet principal	Assist'IA — Application web immobilière
Expérience complémentaire	Innbox — DataLake (alternance)
Période	Août 2025 – Octobre 2026
Auteur	Diana MARGARIAN

Ce dossier présente la conception, le développement, la sécurisation et le déploiement d'Assist'IA, ainsi que l'expérience menée en alternance chez Innbox.

SOMMAIRE

I. Présentation personnelle et du projet en anglais 6
Glossaire 6
II. Présentation du projet 7
II.1 Contexte et problématique 7
II.2 Solution développée 7
II.3 Technologies utilisées 8
III. Cahier des charges 8
III.1 Objectifs du projet 8
III.2 Contexte métier 8
III.3 Exigences fonctionnelles 9
III.4 Exigences techniques 9
III.5 Contraintes temporelles 10
III.6 Analyse des risques 10
IV. Méthodologie et organisation 11
IV.a Gestion de projet 11
IV.b Versioning (Git, stratégie de branches) 12
IV.c DevOps — CI (intégration continue) 12
IV.d DevOps — CD (déploiement continu) 14
V. Conception UI/UX 15
V.a Zoning 16
V.b Wireframes 18
V.c Charte graphique 20
V.d Maquettes graphiques 20
VI. Modélisation de la base de données (Merise) 23
VI.a Introduction à la méthode Merise 23
VI.b Dictionnaire des données 24
VI.c Modèle conceptuel de données (MCD) 26
VI.d Modèle logique de données (MLD) 27
VI.e Modèle physique de données (MPD) 28
VII. Conception de l'application (UML) 30
VII.a Diagrammes de cas d'utilisation 30
VII.b Diagrammes de séquences 31
VII.c Diagrammes de classes UML 33
VIII. Architecture multi-couches 34
VIII.a Modèle MVC (couche logique) 34
VIII.b Architecture n-tiers (tiers physique) 35
VIII.c Séparation des responsabilités 37
IX. Sécurité 38

X. Politique de tests 41
XI. Déploiement et mise en production 43
XI.a Conteneurisation avec Docker 43
XI.b Environnements de déploiement 44
XI.c Stratégies de mise en production 44
XII. Veille technologique 45
XIII. Veille sécurité 45
XIV. Difficultés rencontrées 46
XV. Expérience en alternance — Innbox / DataLake 47
XVI. Conclusion et ouverture 49
XVII. Remerciements 50
XVIII. Annexes 50

LISTE DES FIGURES

Figure IV.I : Tableau Kanban du jalon 2 11
Figure IV.V : Planning prévisionnel 12
Figure IV.II : Script du workflow d'intégration continue 13
Figure IV.III : Exécution du workflow de déploiement continu 14
Figure IV.IV : Script du workflow CD 15
Figure V.I : Zoning de la page d'accueil 16
Figure V.II : Zoning de la fiche bien 17
Figure V.V : Arborescence du site 18
Figure V.III : Wireframe de la page d'accueil 19
Figure V.IV : Wireframe de la fiche bien 20
Figure V.VII : Écran réalisé : catalogue 23
Figure V.VIII : Maquette haute fidélité : accueil 21
Figure V.IX : Maquette haute fidélité : fiche bien 22
Figure VI.I : Les quatre niveaux de la méthode Merise 24
Figure VI.II : Modèle conceptuel de données 27
Figure VI.III : Modèle logique de données 28
Figure VII.I : Diagramme de cas d'utilisation 31
Figure VII.II : Séquence de connexion agent 32
Figure VII.III : Séquence de prise de rendez-vous 32
Figure VII.IV : Diagramme de classes 33
Figure VIII.I : Schéma de l'architecture n-tiers 36
Figure VIII.II : Organisation contrôleurs, services, repositories 37
Figure IX.I : Jeton CSRF du formulaire de connexion 39
Figure IX.II : Activation CSRF sur le pare-feu de connexion 39
Figure IX.III : Limitation des tentatives de connexion 40
Figure X.I : Pipeline CI exécutant les tests PHPUnit 42
Figure XI.I : Dockerfile de l'application 43
Figure XV.I : Architecture du DataLake Innbox 48

LISTE DES ANNEXES

Annexe A1 : Captures de l'application 51
Annexe A2 : Résultats des tests PHPUnit 52
Annexe A3 : Workflows GitHub Actions 53
Annexe A4 : Page de politique de confidentialité 53
Annexe A5 : Fichiers Docker Compose 55
Annexe A6 : Schéma d'architecture DataLake 58
Annexe A7 : Tableaux de bord Grafana 59

I. Présentation personnelle et du projet en anglais

Personal introduction

My name is Diana MARGARIAN, I am a student at IPSSI. During my training, I worked on a full-stack web project called Assist'IA, a real-estate application that helps visitors find properties, chat with an AI assistant, book appointments, and contact the agency. Agents can manage listings, view client conversations, and handle their calendar through a secure back-office. I am particularly interested in web development, software architecture, and DevOps practices. This project allowed me to apply Symfony, Docker, automated testing, and continuous integration in a realistic business context.

Project overview in English

Assist'IA is a Symfony 7.4 web application developed as a training capstone project. It targets a fictional real-estate agency and addresses three user profiles:

- Visitors browse published properties, use an embedded AI chat to refine their search (budget, property type), book appointments, and send contact messages.
- Real-estate agents authenticate to manage their property listings, consult chat histories, and manage availability slots.
- Administrators supervise the entire agency through EasyAdmin (all properties and all slots).

The technical stack includes PHP 8.3, Symfony, Doctrine ORM, MySQL 8, Twig, React (chat widget), Tailwind CSS, Docker, and GitHub Actions for CI/CD. External integrations include the French government address API (data.gouv.fr) and optional Ollama LLM for the chatbot. The application follows a layered MVC architecture, implements security best practices (CSRF, password hashing, login throttling, role-based access), includes 23 automated PHPUnit tests, and supports production deployment via Docker Compose.

Glossaire

Terme	Définition
Kanban	Méthode visuelle : une carte par tâche, déplacée de colonne en colonne.
Merise	Méthode de conception des données : dictionnaire, MCD, MLD, MPD.
MVC	Modèle, vue, contrôleur : séparation des données, de l'écran et des requêtes.
ORM	Doctrine : fait le lien entre les objets PHP et les tables MySQL.
CSRF	Attaque qui force une action au nom d'un utilisateur déjà connecté.
RGPD	Règlement européen sur les données personnelles.
CI / CD	Contrôles automatiques à chaque envoi de code, puis préparation d'une version.

II. Présentation du projet

II.1 Contexte et problématique

Dans le cadre de ma formation à l'IPSSI, j'ai mené un projet fil rouge sur plusieurs mois. Ce projet, nommé Assist'IA, constitue le cœur de ce dossier : il a été conçu pour couvrir l'ensemble des compétences du titre CDA (conception, développement, sécurité, tests, déploiement).

Parallèlement, j'ai effectué une alternance d'un an chez Innbox (self storage) en tant que Junior Data Engineer. Cette expérience professionnelle est présentée en section complémentaire car elle ne couvre pas toutes les compétences du titre, principalement la data et l'automatisation, mais elle enrichit mon profil et montre ma capacité à mener un projet en entreprise.

Avant ce projet, une agence de ce type s'appuie généralement sur des outils dispersés : annonces sur des portails tiers, prises de contact par téléphone ou e-mail, et peu de visibilité sur les intentions des visiteurs. Les agents manquent d'un outil unique pour publier leurs biens, suivre les demandes et organiser leurs rendez-vous.

La problématique centrale est la suivante : comment offrir aux visiteurs une expérience de recherche fluide, tout en donnant aux agents un espace métier sécurisé pour gérer annonces, conversations et planning ?

II.2 Solution développée

Assist'IA répond à cette problématique par une application web monolithique Symfony, organisée en deux zones.

Espace public (visiteur), accessible sans authentification :

- Page d'accueil avec assistant IA intégré et aperçu des biens récents
- Catalogue public des annonces publiées (/biens)
- Prise de rendez-vous sur créneaux disponibles (/rdv)
- Formulaire de contact (/contact)
- Page de politique de confidentialité (/politique-confidentialite)

Espace agent, protégé par authentification :

- Tableau de bord avec statistiques (/agent)
- Gestion CRUD des biens (/property)
- Historique des conversations clients (/agent/chats)
- Calendrier et gestion des créneaux (/agent/calendrier)

Espace administrateur, rôle ROLE_ADMIN : back-office EasyAdmin (/admin) pour la supervision globale.

Des API JSON internes alimentent les composants dynamiques : POST /api/chat (assistant IA) et GET /api/address/search (autocomplétion d'adresse, agents uniquement).

II.3 Technologies utilisées

Domaine	Technologie	Version / remarque
Langage	PHP	8.3 (Docker), ≥ 8.2 (composer)
Framework	Symfony	7.4.*
ORM	Doctrine	3.6
Base de données	MySQL	8.0
Templates	Twig	3.x
CSS	Tailwind CSS	CDN, utility-first
JavaScript	React	19.x — widget chat
JS framework	Stimulus + Turbo	Symfony UX
Admin	EasyAdmin	4.28
Conteneurisation	Docker, nginx, php-fpm	—
CI/CD	GitHub Actions	CI + CD
Tests	PHPUnit	11.5 — 23 tests
IA (optionnel)	Ollama + llama3.2	Repli sur des règles métier
API externe	api-adresse.data.gouv.fr	Adresses France

III. Cahier des charges

III.1 Objectifs du projet

Les objectifs principaux définis dans le cahier des charges technique et les livrables IPSSI sont :

- Permettre aux visiteurs de découvrir les biens, d'interagir avec un assistant IA et de prendre rendez-vous.
- Permettre aux agents de gérer leurs annonces et leur activité commerciale.
- Garantir la sécurité des accès et des données (authentification, rôles, RGPD).
- Assurer la qualité du code via des tests automatisés et l'intégration continue.
- Préparer le déploiement en environnement conteneurisé (Docker).

III.2 Contexte métier

L'application simule une agence immobilière fictive Assist'IA. Les acteurs métier sont :

Acteur	Besoin
Visiteur	Trouver un bien, obtenir des conseils, prendre rendez-vous
Agent immobilier	Publier et gérer ses annonces, voir les chats, gérer son calendrier
Directeur / admin	Superviser l'ensemble de l'agence
Formateur	Valider la conformité au référentiel CDA

Les parties prenantes du projet sont les suivantes.

Partie prenante	Rôle	Attente
Diana MARGARIAN	Conçoit et développe	Une application complète et défendable
Visiteur	Consulte sans compte	Trouver un bien et prendre rendez-vous
Agent immobilier	Espace authentifié	Gérer annonces, chats et calendrier
Formateur IPSSI	Évalue le dossier	Respect du référentiel CDA

III.3 Exigences fonctionnelles

Visiteur. Consulter la liste des biens publiés uniquement ; voir le détail d'un bien (prix, ville, surface, photo) ; discuter avec l'assistant IA (budget, typologie T1/T2) ; réserver un créneau de rendez-vous ; envoyer un message via le formulaire de contact ; consulter la politique de confidentialité.

Agent. Se connecter et se déconnecter ; réinitialiser son mot de passe (e-mail + jeton) ; créer, modifier, supprimer ses biens (brouillon ou publié) ; bénéficier de l'autocomplétion d'adresse ; consulter l'historique des conversations par session ; visualiser et créer des créneaux ; accéder à un tableau de bord synthétique.

Administrateur. Accéder à EasyAdmin (tous les biens, tous les créneaux) et bénéficier de tous les droits agents.

Automatisations. Génération des créneaux (lundi–vendredi, 10 h et 14 h, 14 jours) via la commande app:ensure-availability-slots ; chargement de données de démonstration via app:load-demo-data.

III.4 Exigences techniques

- Architecture MVC Symfony, code organisé en services
- Base de données relationnelle MySQL avec migrations Doctrine
- Interface responsive (mobile-first avec Tailwind)
- API REST interne pour le chat et les adresses
- Docker pour un environnement de développement reproductible

- Tests PHPUnit (unitaires et fonctionnels)
- Pipeline CI sur chaque push vers main
- Pipeline CD sur les tags versionnés (v*)
- Conformité RGPD (page dédiée, mentions)

III.5 Contraintes temporelles

Le projet a été réalisé sur environ cinq à six mois. Le calendrier de formation imposait des points de livraison. Le tableau ci-dessous situe ces échéances. La façon dont le travail a été piloté au quotidien est décrite dans la partie gestion de projet.

Échéance	Livrables principaux	Tag Git
Échéance 1–2	Setup Symfony, Docker, premières entités	—
Échéance 3	Catalogue, authentification agent	—
Échéance 4	Chat IA, contact, rendez-vous	—
Échéance 5	CI, tests, sécurisation	v0.5-beta
Échéance 6	Production, CD, RGPD, réinitialisation du mot de passe, e-mails de RDV	v1.0

Critères de succès

- Les parcours visiteur, agent et administrateur fonctionnent.
- Les 23 tests PHPUnit passent dans la CI.
- L'application se lance avec Docker, en développement et en production.
- Les accès sont séparés par rôles, et les données personnelles sont expliquées sur la page RGPD.

III.6 Analyse des risques

Ces risques ont été identifiés avant le développement. La parade est ce qui a été prévu pour les limiter.

Risque	Probabilité	Impact	Parade
Ollama indisponible	Moyenne	Moyen	Réponse par règles métier si le modèle ne répond pas
Deux réservations du même créneau	Faible	Élevé	Contrôle du créneau avant enregistrement

Risque	Probabilité	Impact	Parade
Fuite de mots de passe ou d'e-mails	Faible	Élevé	Hachage, rôles, secrets hors du dépôt
Écart entre le poste et la production	Moyenne	Moyen	Même image Docker partout

IV. Méthodologie et organisation

IV.a Gestion de projet

J'ai piloté Assist'IA au quotidien avec un tableau Kanban. Chaque sujet est une carte. Le tableau a quatre colonnes : Backlog, À faire, En cours, Terminé. Une carte quitte le backlog quand elle est prête à être planifiée. Elle passe dans À faire, puis dans En cours le jour où je la développe. Elle n'entre dans Terminé que lorsque le livrable est posé. Je ne garde dans En cours que ce que je suis capable de finir.

La figure IV.I est le tableau tel qu'il était au jalon 2.

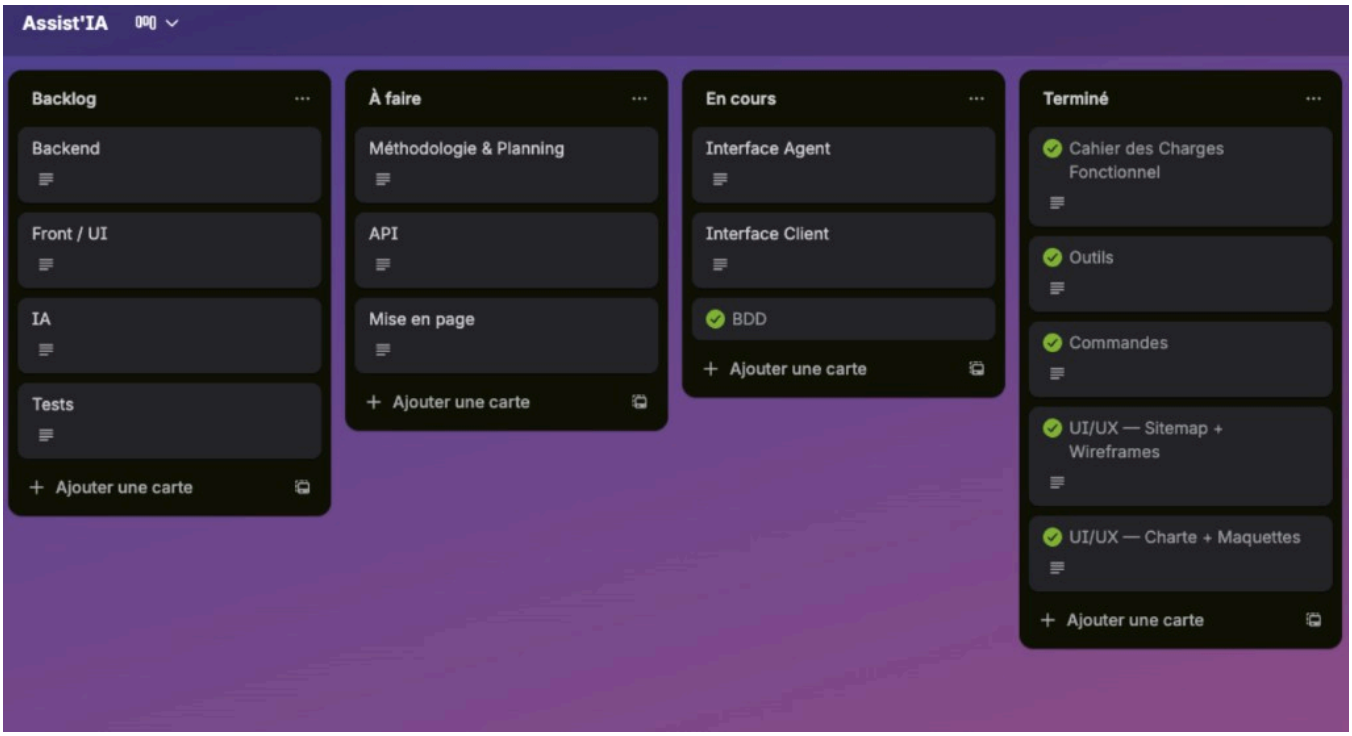


Figure IV.I : Tableau Kanban du jalon 2

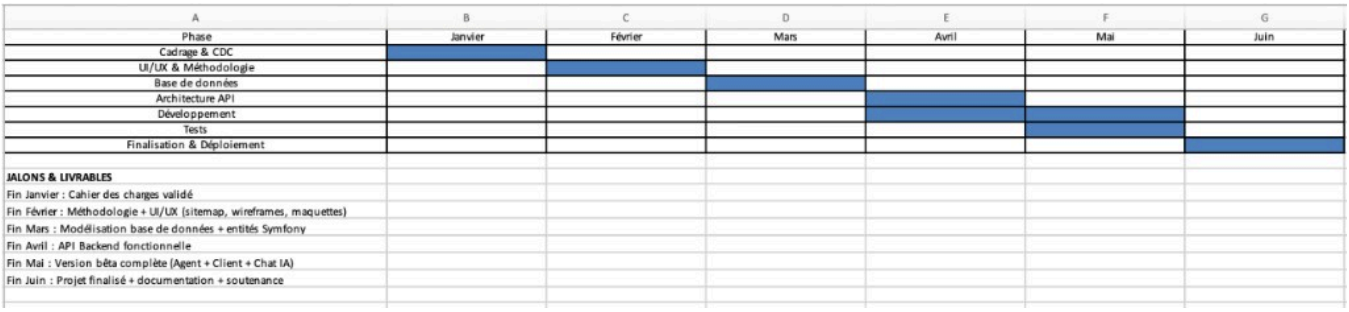
Journée type

Ce matin-là, j'ai ouvert le tableau avant de coder. J'ai d'abord lu la colonne En cours : Interface Agent et Interface Client y étaient ouvertes. La carte BDD était cochée, la base était en place : je l'ai glissée dans Terminé, avec le cahier des charges, les commandes et les livrables UI/UX déjà rangés. Ensuite seulement j'ai regardé À faire. J'ai pris la carte API, je l'ai déplacée dans En

cours, et j'ai travaillé dessus. Le backlog (Backend, Front / UI, IA, Tests) n'a pas bougé : ces cartes attendent que En cours se vide.

C'est ce geste — lire En cours, finir une carte, n'en ouvrir une nouvelle que depuis À faire — que j'ai répété tout au long du projet.

Le diagramme de Gantt ci-dessous situe les phases sur six mois. Il donne le calendrier. Il ne remplace pas le tableau : au jour le jour, c'est le déplacement des cartes qui dit où en est le travail.



Phase	Janvier	Février	Mars	Avril	Mai	Juin
Cadrage & CDC						
UI/UX & Méthodologie						
Base de données						
Architecture API						
Développement						
Tests						
Finalisation & Déploiement						

JALONS & LIVRABLES

Fin Janvier : Cahier des charges validé

Fin Février : Méthodologie + UI/UX (sitemap, wireframes, maquettes)

Fin Mars : Modélisation base de données + entités Symfony

Fin Avril : API Backend fonctionnelle

Fin Mai : Version bêta complète (Agent + Client + Chat IA)

Fin Juin : Projet finalisé + documentation + soutenance

Figure IV.V : Planning prévisionnel

IV.b Versioning (Git, stratégie de branches)

Le code source est hébergé sur GitHub. La branche principale main constitue la référence stable. Pratiques adoptées :

- Commits atomiques avec messages explicites
- Tags de release : v0.5-beta, puis v1.0
- Pull requests contrôlées par la CI

Commandes type pour une release :

```
git tag -a v1.0 -m "Version finale"
git push origin v1.0
```

IV.c DevOps — CI (intégration continue)

Le workflow .github/workflows/ci.yml s'exécute à chaque push sur main et sur les pull requests. La capture ci-dessous est le script tel qu'il est versionné dans le dépôt.

.github/workflows/ci.yml

```yaml
name: CI Symfony

on:
  push:
    branches: [ "main" ]
  pull_request:

jobs:
  symfony:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: "8.3"
          tools: composer
          coverage: none
          extensions: pdo_sqlite, intl

      - name: Validate composer.json
        run: composer validate --no-interaction --no-check-publish

      - name: Install dependencies
        run: composer install --no-interaction --prefer-dist

      - name: Symfony console check
        run: php bin/console --version

      - name: Lint YAML
        run: php bin/console lint:yaml config --parse-tags

      - name: Lint Twig
        run: php bin/console lint:twig templates

      - name: Lint container
        run: php bin/console lint:container

      - name: Security audit
        run: composer audit --no-interaction || true

      - name: Prepare test environment
        run: mkdir -p var && php bin/console doctrine:schema:create --env=test --no-interaction

      - name: Run PHPUnit
        run: php bin/phpunit
```

Figure IV.II : Script du workflow d'intégration continue

Étapes du pipeline :

- Checkout du code
- Installation de PHP 8.3 avec les extensions pdo_sqlite et intl
- composer validate et composer install
- Vérification Symfony (php bin/console --version)
- Lint YAML, Twig et conteneur Symfony
- composer audit (audit de sécurité des dépendances)

- Création du schéma de test (SQLite)
- Exécution de PHPUnit (23 tests)

Ce pipeline garantit qu'aucune régression majeure n'est introduite sur la branche principale.

IV.d DevOps — CD (déploiement continu)

Le déploiement continu complète l'intégration continue. La CI vérifie la qualité du code à chaque push. Le CD prépare une version déployable lorsqu'un tag de version est créé (par exemple v1.0). Le workflow .github/workflows/cd.yml se déclenche automatiquement sur les tags v*, et manuellement via workflow_dispatch.

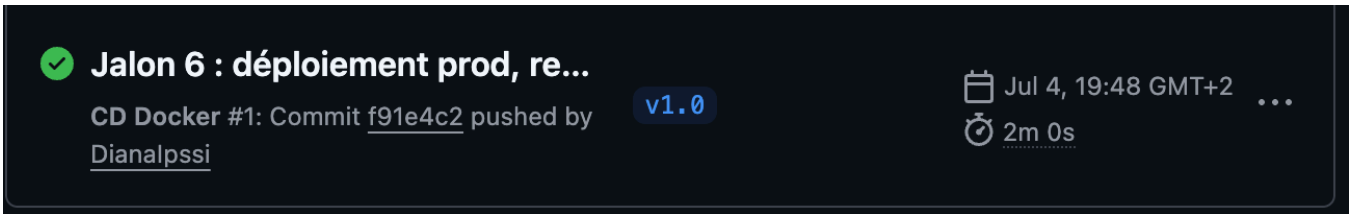


Figure IV.III : Exécution du workflow de déploiement continu

Étapes du pipeline CD : checkout du code source ; mise en place de Docker Buildx ; build de l'image PHP (assist-ia-php:latest) à partir du Dockerfile ; validation de docker-compose.yml ; validation de docker-compose.prod.yml.

.github > workflows > ! cd.yml

```yaml
name: CD Docker

on:
  push:
    tags:
      - 'v*'
  workflow_dispatch:

jobs:
  build-image:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build PHP image
        uses: docker/build-push-action@v6
        with:
          context: .
          file: ./Dockerfile
          push: false
          tags: assist-ia-php:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Validate docker-compose
        run: docker compose -f docker-compose.yml config

      - name: Validate docker-compose prod
        run: docker compose -f docker-compose.prod.yml config

```

Figure IV.IV : Script du workflow CD

V. Conception UI/UX

La conception des écrans a commencé par le zoning, puis par des wireframes basse fidélité, avant la charte graphique et les maquettes. L'objectif était de figer la place de chaque bloc (identité, navigation, contenu, action) avant d'écrire le HTML.

V.a Zoning

Le zoning de la page d'accueil reprend une grille classique. Le logo et l'encart publicitaire occupent la première bande. La navigation est pleine largeur. Le corps de page est coupé en deux colonnes : l'édito (promesse de l'agence) en face des actualités (assistant et biens récents), puis la présentation des produits en face du bloc de téléchargement / prise de rendez-vous. Le pied de page ferme la page.

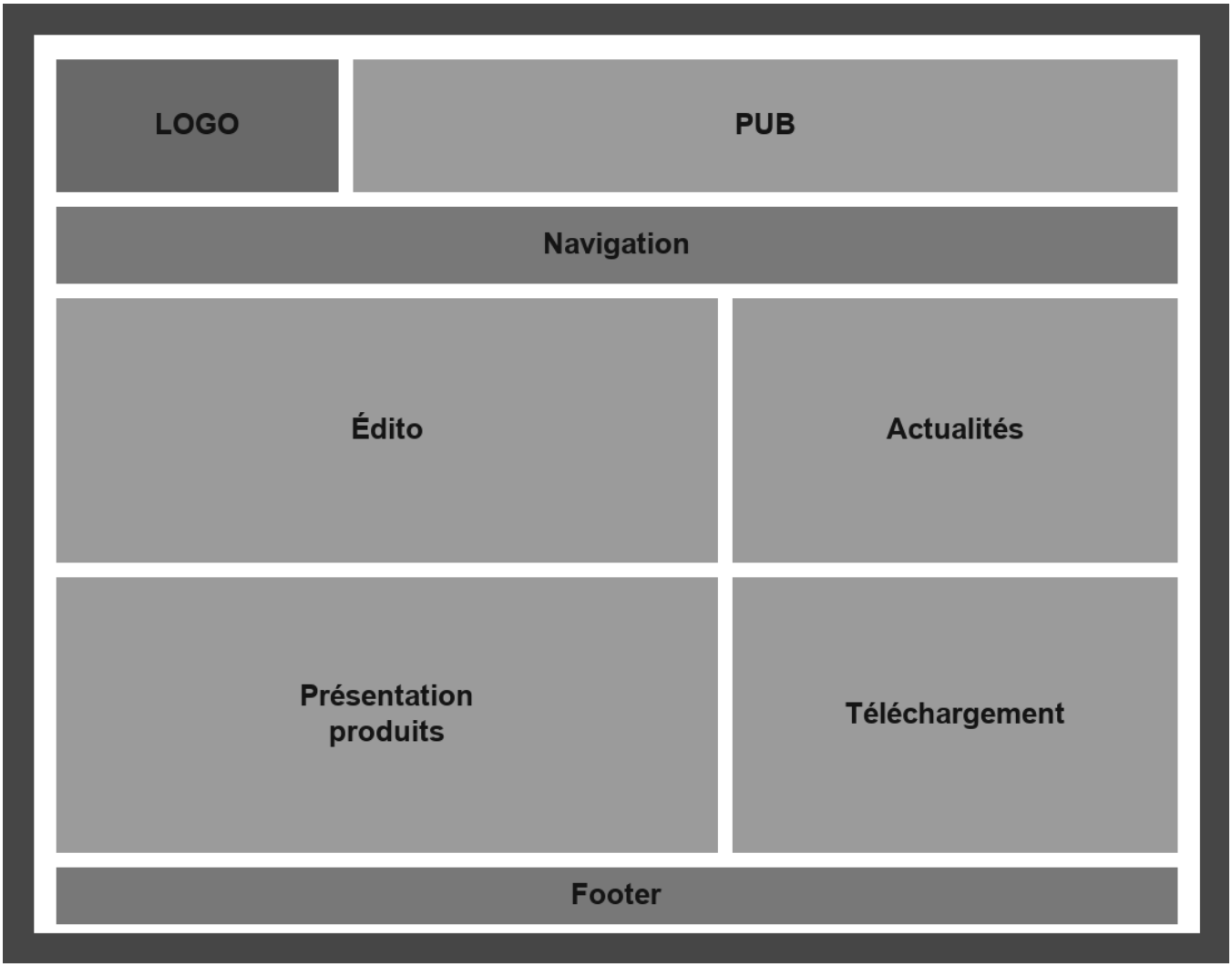


Figure V.I : Zoning de la page d'accueil

La fiche bien garde le même en-tête et le même pied de page. La colonne de gauche est réservée à la photo, la colonne de droite au prix, à la ville et à l'action « Prendre RDV ». Une bande de biens similaires précède le footer.

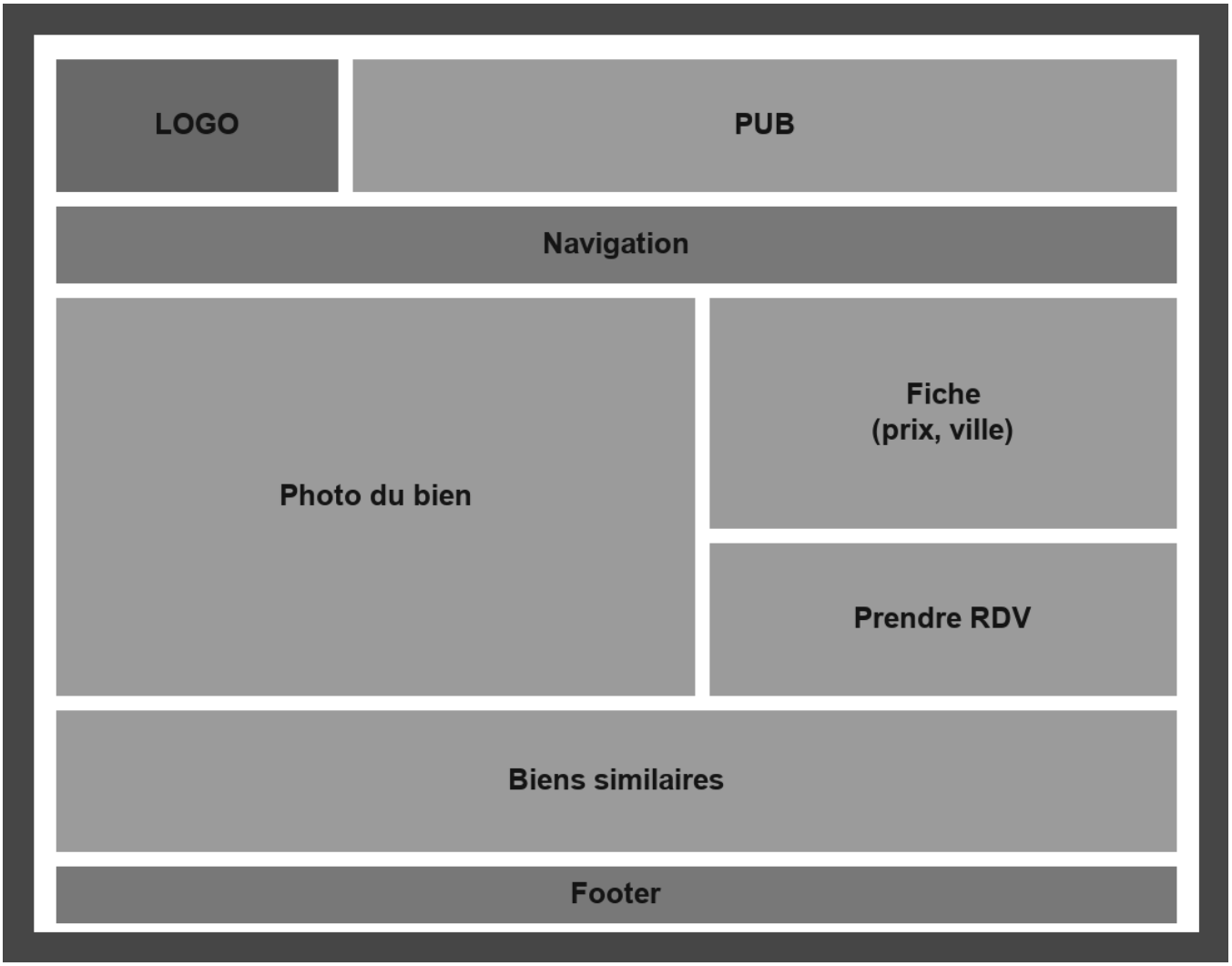


Figure V.II : Zoning de la fiche bien

La navigation publique (header dans base.html.twig) propose Accueil, Biens, Prendre RDV et Contact. Le footer donne accès à la page Confidentialité et à l'espace agent. L'arborescence relie ces écrans : le public reste à gauche, l'espace agent et l'administration ne s'ouvrent qu'après connexion.

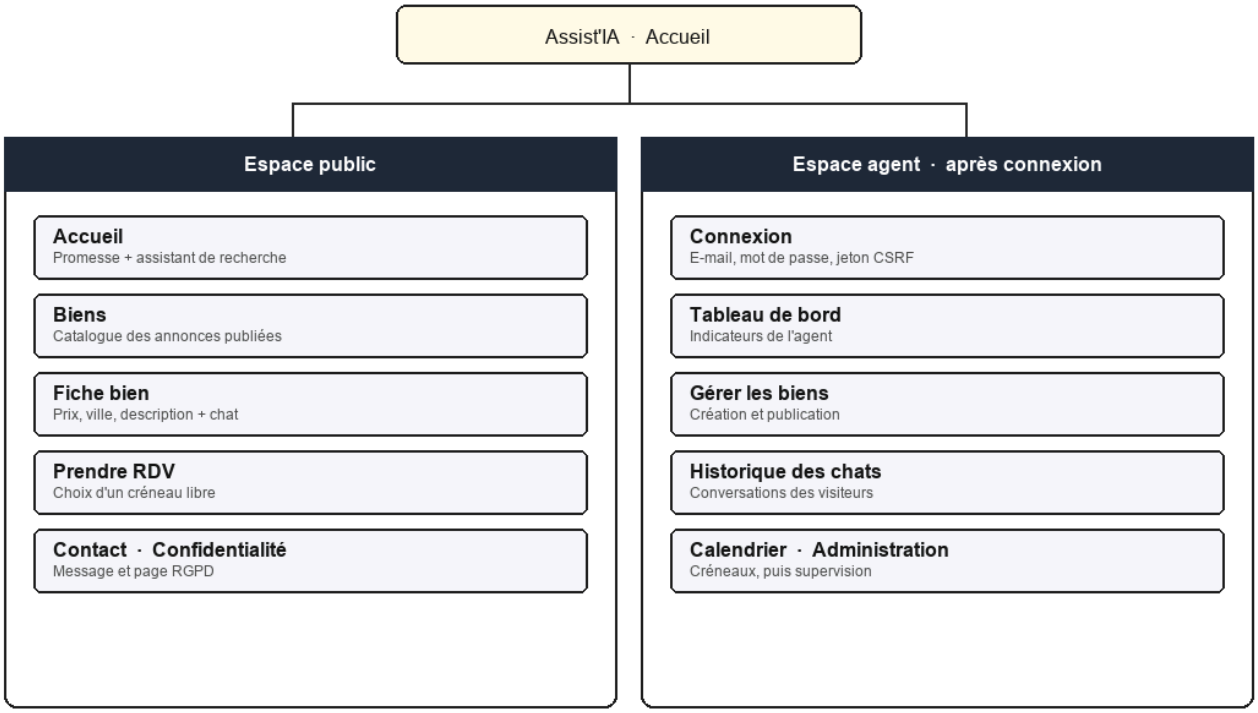


Figure V.V : Arborescence du site

V.b Wireframes

Les wireframes précisent la structure sans couleur : fond gris, cartes blanches, texte noir. Ils reprennent les deux pages réellement construites. L'accueil place le texte et les actions à gauche, l'assistant de recherche à droite, puis les biens récents. La fiche bien empile la photo, le prix, la ville, la surface et la description. Le chat reste en bas à droite.

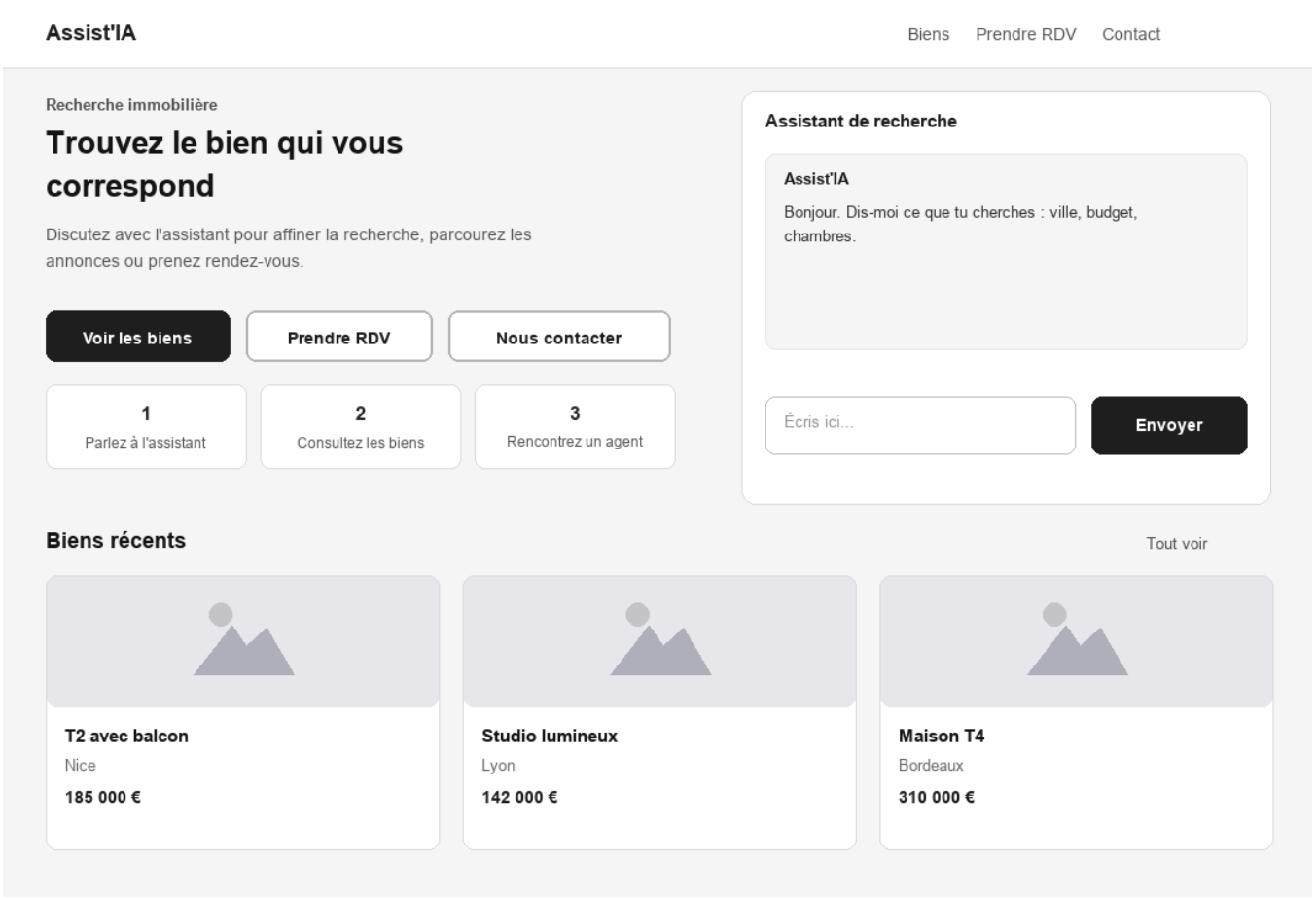


Figure V.III : Wireframe de la page d'accueil

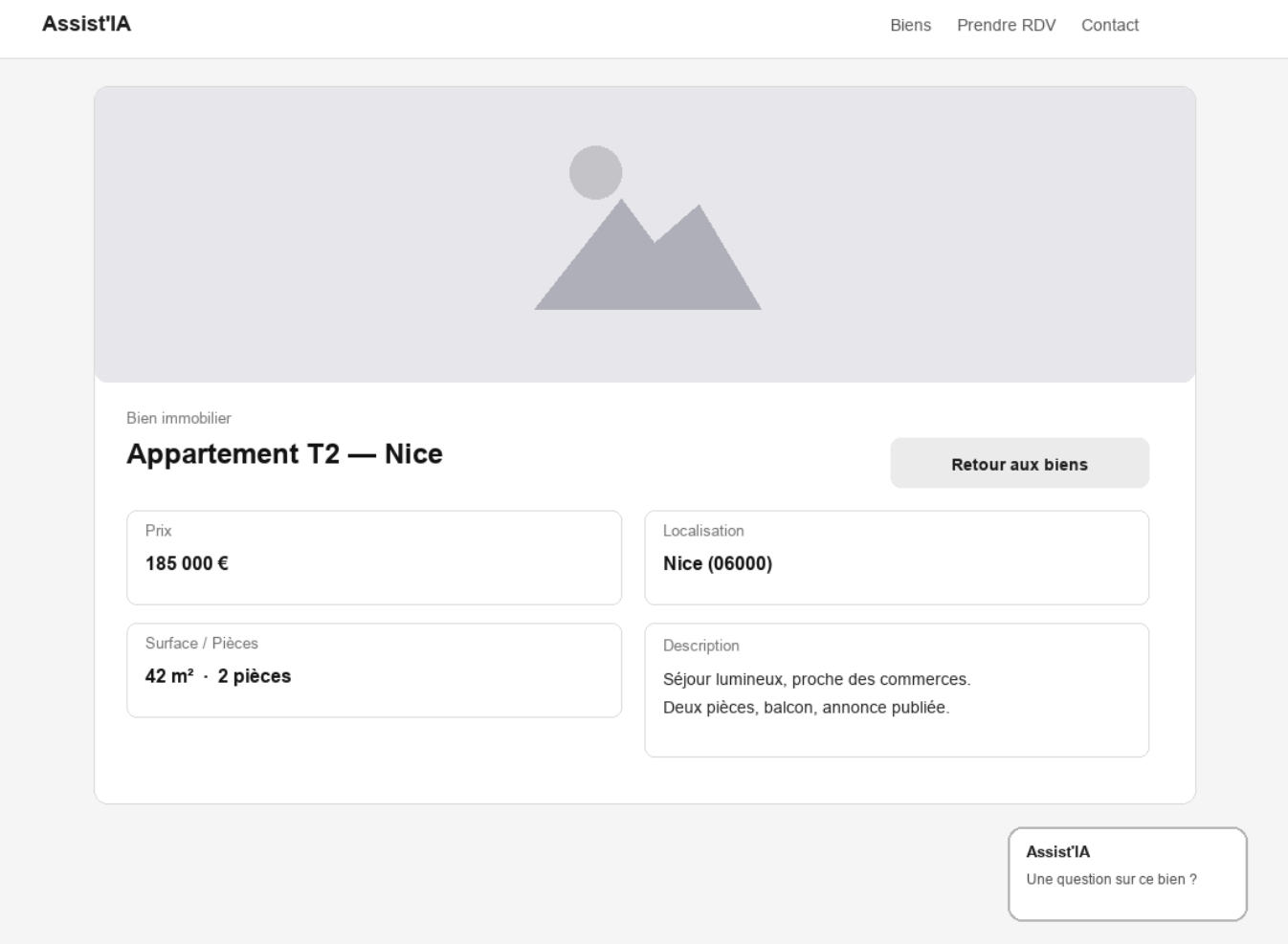


Figure V.IV : Wireframe de la fiche bien

V.c Charte graphique

Élément	Choix
Couleur principale	Rose corail (rose-400, #f472b6)
Fond	Gris clair (bg-gray-50)
Cartes	Blanc, bords arrondis (rounded-2xl), ombre légère
Typographie	System-ui, titres en gras
Boutons primaires	Fond rose, texte blanc, survol rose-500
Liens	Rose, soulignés au survol

Cette charte est appliquée sur toutes les pages Twig via les classes Tailwind.

V.d Maquettes graphiques (desktop et mobile)

En desktop, le contenu est limité à max-w-6xl et centré, avec des grilles multi-colonnes (lg:grid-cols-2, md:grid-cols-3). En mobile, les colonnes s'empilent (flex-col), la navigation se compacte

et les cartes de biens passent en liste verticale.

Page	Template	Particularités
Accueil	home/index.html.twig	Chat React embarqué, parcours en 3 étapes
Catalogue	public/property/index.html.twig	Cartes responsive
Contact	public/contact/index.html.twig	Formulaire Symfony stylisé
Rendez-vous	public/appointment/index.html.twig	Sélection d'un créneau
Login	security/login.html.twig	CSRF, lien mot de passe oublié
Tableau de bord	agent/dashboard/index.html.twig	Indicateurs, tableau, actions

Composants dynamiques : le widget chat React (assets/app.js) envoie les messages de façon asynchrone et mémorise la session ; l'autocomplétion d'adresse interroge l'API avec un délai de 300 ms.

Les maquettes haute fidélité appliquent la charte : rose sur les actions, fond gris, cartes blanches. L'accueil est la page livrée. La fiche bien reprend le même gabarit, avec le chat en bulle.

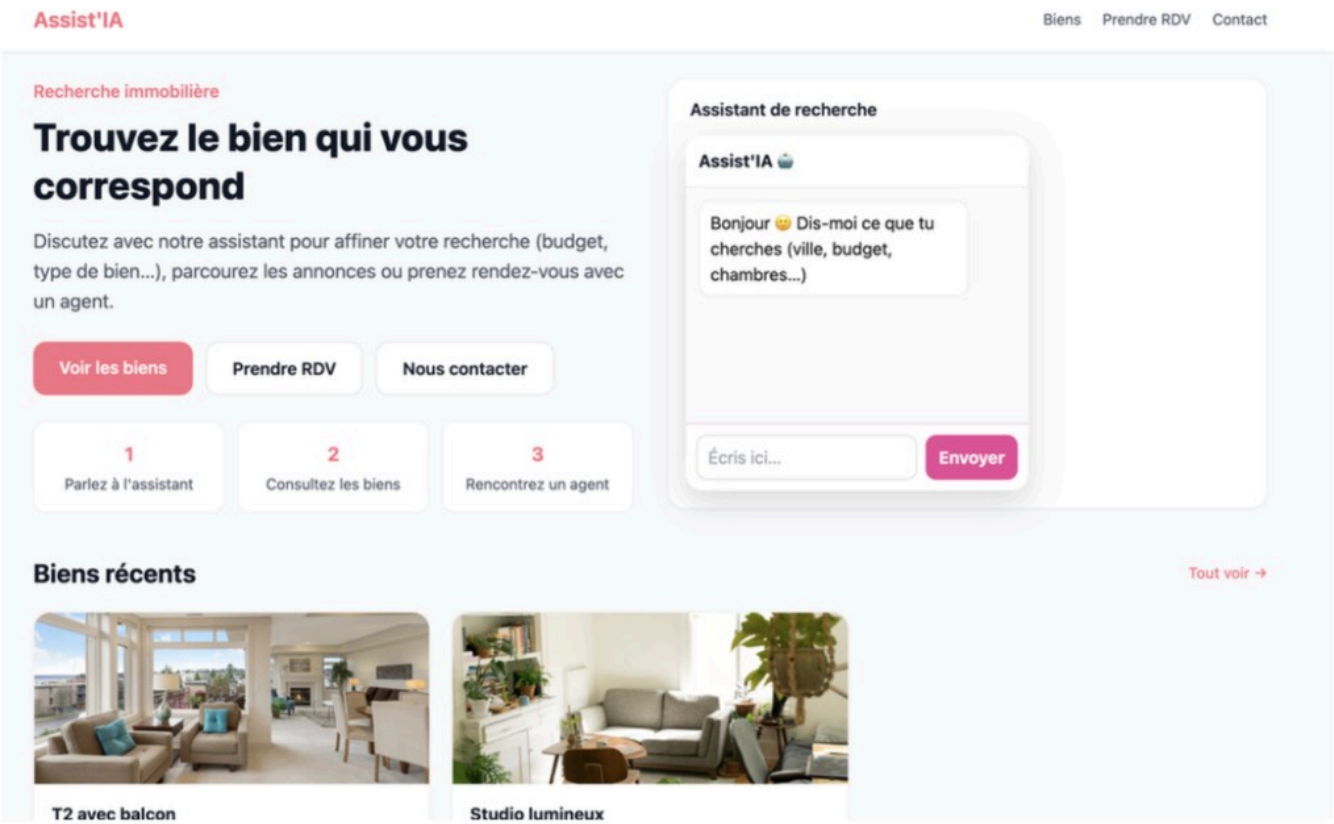


Figure V.VIII : Maquette haute fidélité : accueil

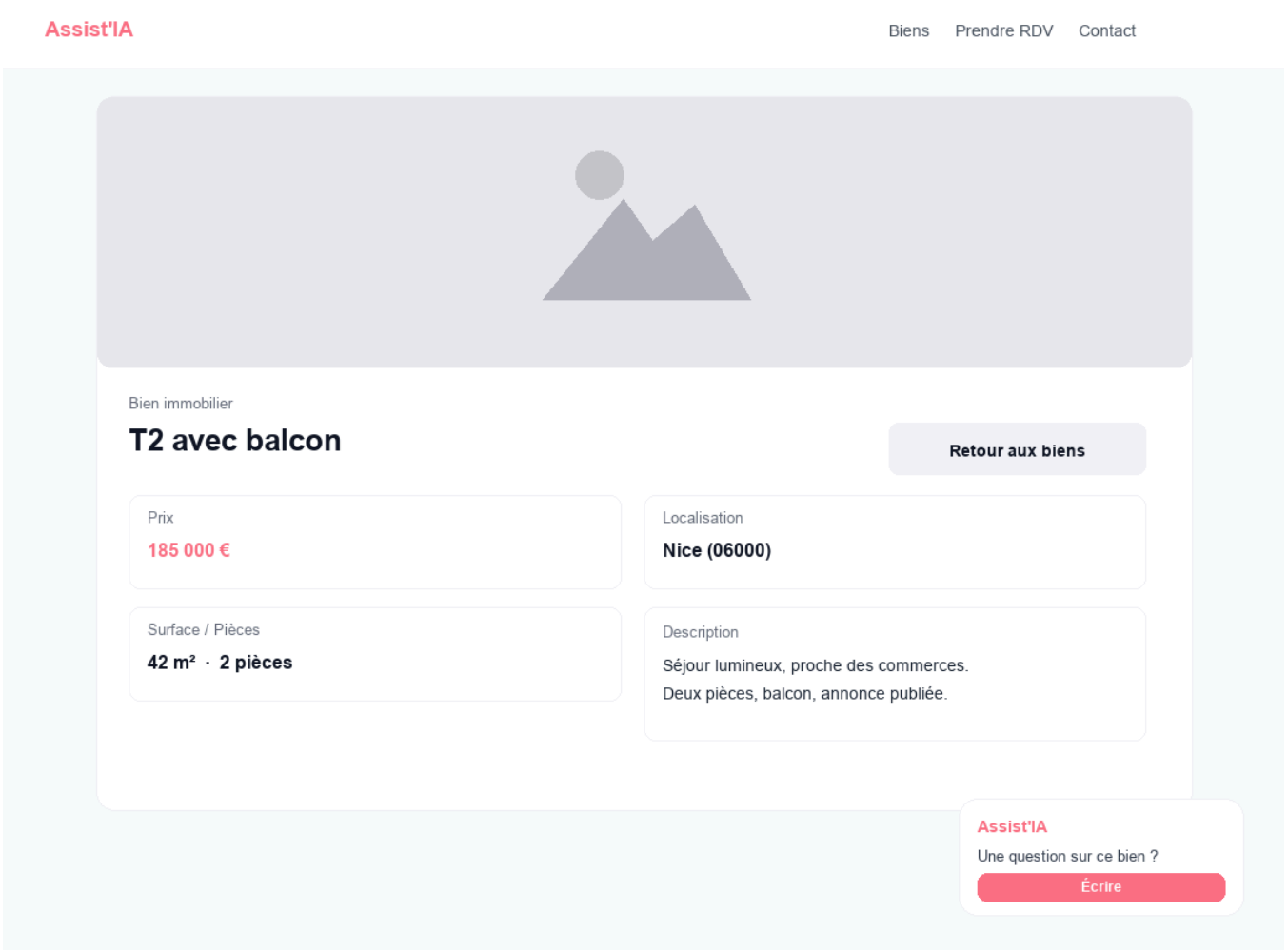


Figure V.IX : Maquette haute fidélité : fiche bien

Le catalogue livré est en annexe, avec les autres captures. Un extrait est repris ici.

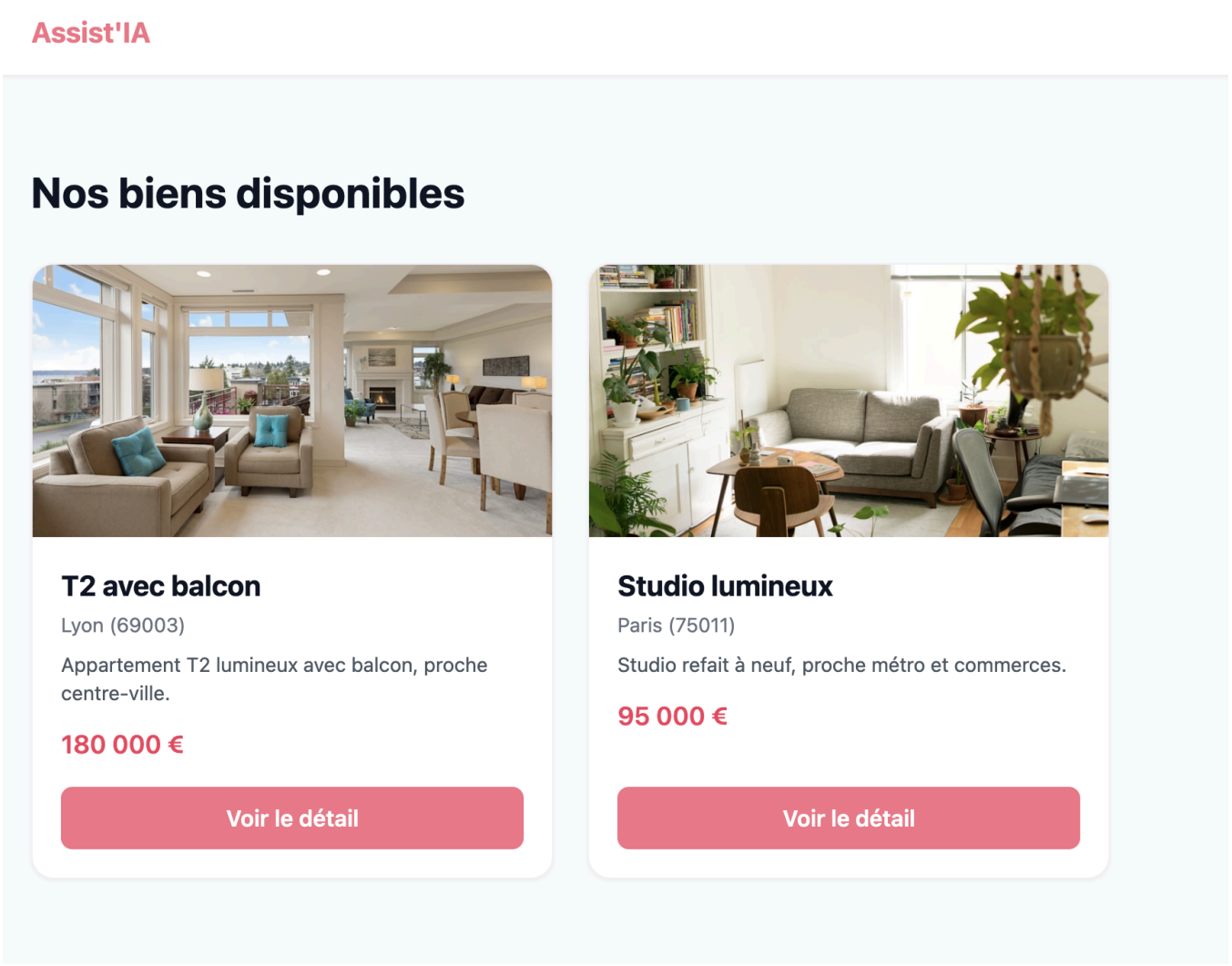


Figure V.VII : Écran réalisé : catalogue

VI. Modélisation de la base de données (Merise)

VI.a Introduction à la méthode Merise

La méthode Merise est une méthode française d'analyse et de conception des systèmes d'information. Elle permet de modéliser les données avant de coder, afin d'éviter les erreurs de structure et de garantir la cohérence de la base.

Dans Assist'IA, j'ai appliqué les quatre niveaux classiques.

1. **Dictionnaire des données.** Liste exhaustive des informations manipulées. Chaque donnée a un code mnémonique, une désignation, un type, une taille et une remarque.
2. **MCD (modèle conceptuel de données).** Entités, associations et cardinalités. À ce stade, on ne parle pas encore de tables SQL.
3. **MLD (modèle logique de données).** Transformation du MCD en relations : clé primaire, clé étrangère, tables.
4. **MPD (modèle physique de données).** Implémentation MySQL via les migrations Doctrine.

Intérêt pour Assist'IA : anticiper les liens entre agent, bien et rendez-vous avant de coder ; faire évoluer le schéma (message de chat, jeton de réinitialisation) sans casser l'existant ; respecter l'intégrité référentielle (un rendez-vous sans agent ou sans créneau est impossible).

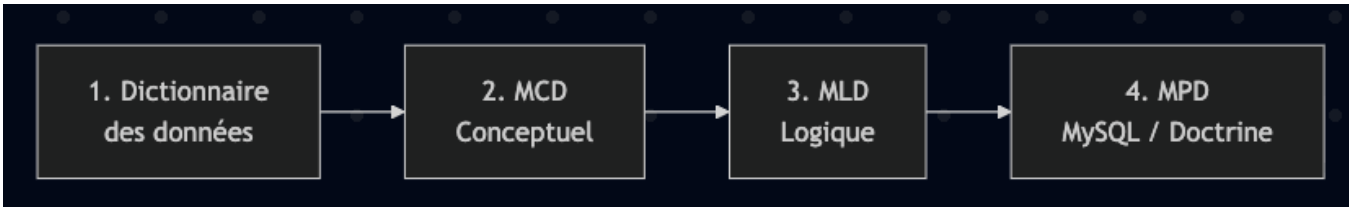


Figure VI.I : Les quatre niveaux de la méthode Merise

VI.b Dictionnaire des données

Le dictionnaire suit le formalisme habituel. La colonne Type ne contient que les familles N (numérique), A (alphabétique), AN (alphanumérique) et Date. La longueur est portée par la colonne Taille. La dernière colonne est une remarque, pas une description libre du champ.

Code mnémonique	Désignation	Type	Taille	Remarque
Agent				
id_a	Identifiant numérique d'un agent	N		
email_a	Adresse e-mail d'un agent	AN	180	Unique, identifiant de connexion
nom_a	Nom complet d'un agent	A	255	
mdp_a	Mot de passe d'un agent	AN	255	Empreinte, jamais en clair
roles_a	Rôles d'un agent	AN	255	ROLE_AGENT ou ROLE_ADMIN
date_creation_a	Date de création du compte	Date	19	Au format AAAA-MM-JJ hh:mm:ss
token_a	Jeton de réinitialisation	AN	64	Nul si aucune demande en cours
token_exp_a	Expiration du jeton	Date	19	Valable 1 heure
Bien				
id_b	Identifiant numérique d'un bien	N		
titre_b	Titre d'une annonce	AN	255	

Code mnémonique	Désignation	Type	Taille	Remarque
prix_b	Prix d'un bien en euros	N		
desc_b	Description d'un bien	AN	1000	
ville_b	Ville du bien	A	120	
cp_b	Code postal du bien	AN	20	
surface_b	Surface du bien en m²	N		
pieces_b	Nombre de pièces	N		
chambres_b	Nombre de chambres	N		
statut_b	Statut de publication	A	30	draft ou published
image_b	Adresse de la photo	AN	500	URL
Créneau				
id_c	Identifiant numérique d'un créneau	N		
debut_c	Début du créneau	Date	19	Au format AAAA-MM-JJ hh:mm:ss
fin_c	Fin du créneau	Date	19	Au format AAAA-MM-JJ hh:mm:ss
dispo_c	Créneau encore libre	A	1	O = oui, N = non
note_c	Note sur le créneau	AN	255	
Rendez-vous				
id_r	Identifiant numérique d'un rendez-vous	N		
nom_r	Nom du client	A	120	
email_r	Adresse e-mail du client	AN	180	
tel_r	Numéro de téléphone du client	AN	30	
msg_r	Message laissé par le client	AN	1000	
date_r	Date de la demande	Date	19	Au format AAAA-MM-JJ hh:mm:ss

Code mnémonique	Désignation	Type	Taille	Remarque
Message				
id_m	Identifiant numérique d'un message	N		
session_m	Identifiant de la conversation	AN	64	Pas de clé étrangère
role_m	Rôle de l'émetteur	A	20	user ou assistant
contenu_m	Texte du message	AN	1000	
nom_m	Nom du visiteur	A	120	
email_m	Adresse e-mail du visiteur	AN	180	
date_m	Date du message	Date	19	Au format AAAA-MM-JJ hh:mm:ss

Règles de gestion

Réf.	Règle
RG01	Un agent est identifié par une adresse e-mail unique.
RG02	Un bien est porté par un seul agent. Un agent peut en publier plusieurs.
RG03	Un créneau appartient à un agent.
RG04	Un rendez-vous occupe exactement un créneau, et un créneau pris n'est plus libre.
RG05	Un message de chat n'a pas de clé étrangère : il est regroupé par session.
RG06	Le mot de passe n'est jamais stocké en clair.

VI.c Modèle conceptuel de données (MCD)

Le MCD ci-dessous est dessiné dans le formalisme de Looping et de jMerise : entité en rectangle, identifiant souligné, association en ovale, cardinalité à côté de l'entité concernée. La cardinalité se lit ainsi : elle indique combien de fois une occurrence de l'entité participe à l'association.

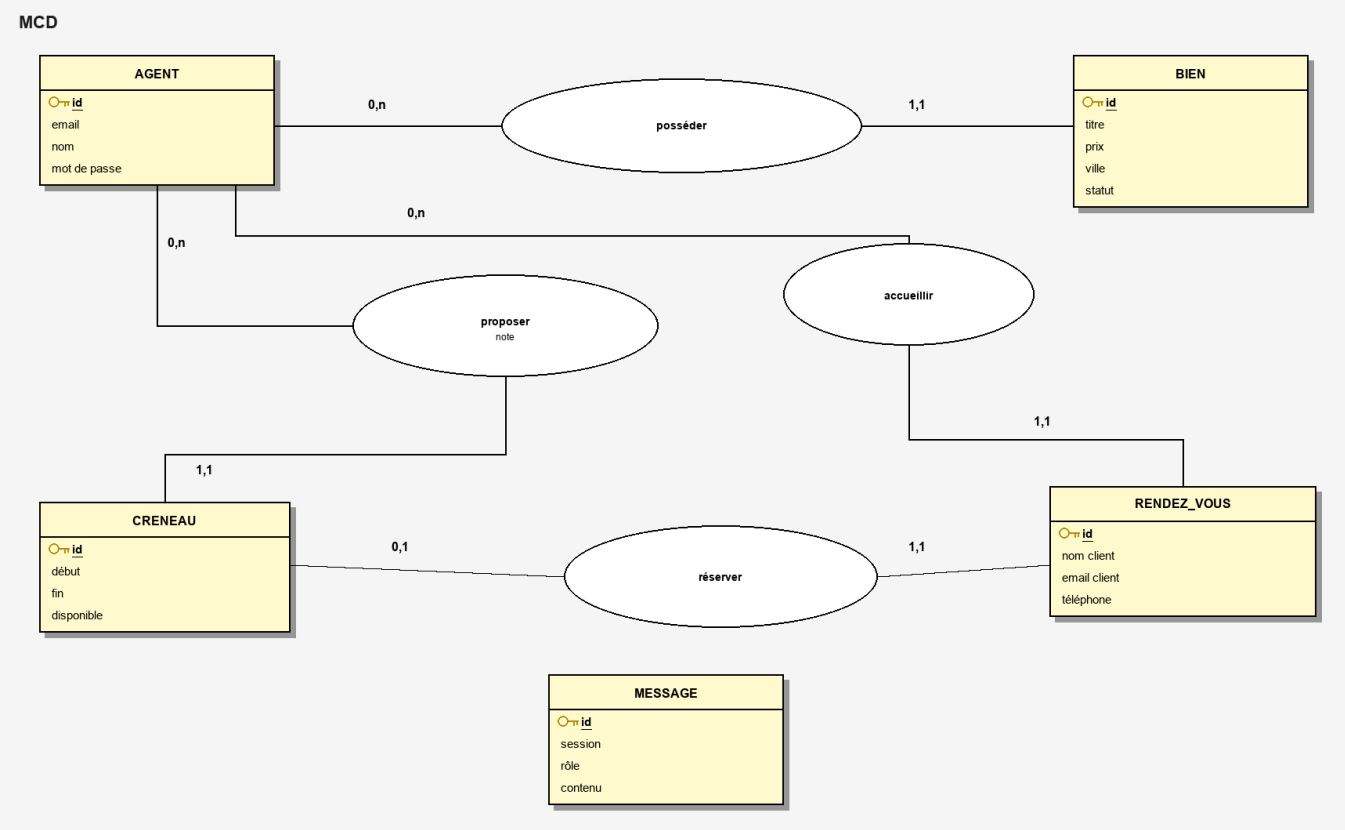


Figure VI.II : Modèle conceptuel de données

- posséder : un agent possède 0 à n biens (0,n) ; un bien est porté par exactement un agent (1,1).
- proposer : un agent propose 0 à n créneaux (0,n) ; un créneau appartient à exactement un agent (1,1). L'association porte la note éventuelle du créneau.
- accueillir : un agent accueille 0 à n rendez-vous (0,n) ; un rendez-vous concerne exactement un agent (1,1).
- réserver : un créneau est lié à 0 ou 1 rendez-vous (0,1) ; un rendez-vous occupe exactement un créneau (1,1).
- MESSAGE est une entité sans association. Les messages d'une même conversation sont regroupés par l'attribut session, sans clé étrangère.

VI.d Modèle logique de données (MLD)

agent (id PK, email UK, full_name, password, roles, created_at, reset_token, reset_token_expires_at)

property (id PK, agent_id FK → agent, title, price, description, city, postal_code, surface, rooms, bedrooms, status, draft, image_url)

availability_slot (id PK, agent_id FK → agent, start_at, end_at, is_available, note)

appointment (id PK, agent_id FK → agent, availability_slot_id FK → availability_slot, client_name, client_email, client_phone, message, created_at)

chat_message (id PK, session_id INDEX, role, content, client_name, client_email, created_at)

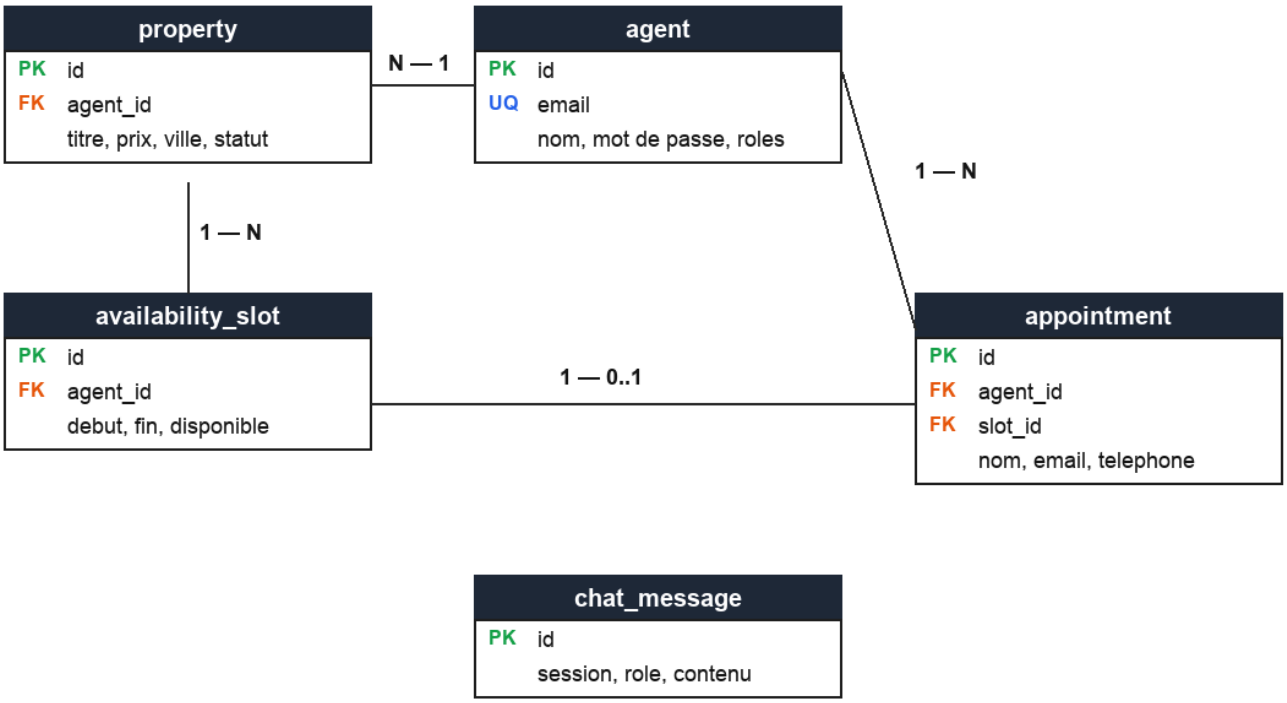


Figure VI.III : Modèle logique de données

VI.e Modèle physique de données (MPD)

Le MPD est le schéma MySQL 8 obtenu après les 13 migrations Doctrine. Les tests utilisent SQLite en mémoire. L'extrait ci-dessous est le script du schéma final : clés primaires, clés étrangères et index, tels que les entités les décrivent.

```
CREATE TABLE agent (
  id INT AUTO_INCREMENT NOT NULL,
  email VARCHAR(180) NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  password VARCHAR(255) NOT NULL,
  roles JSON NOT NULL,
  created_at DATETIME NOT NULL,
  reset_token VARCHAR(64) DEFAULT NULL,
  reset_token_expires_at DATETIME DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE INDEX UNIQ_AGENT_EMAIL (email)
);

CREATE TABLE property (
  id INT AUTO_INCREMENT NOT NULL,
  agent_id INT DEFAULT NULL,
  title VARCHAR(255) NOT NULL,
  price INT NOT NULL,
  description LONGTEXT DEFAULT NULL,
  city VARCHAR(120) DEFAULT NULL,
  postal_code VARCHAR(20) DEFAULT NULL,
  surface INT DEFAULT NULL,
  rooms INT DEFAULT NULL,
  bedrooms INT DEFAULT NULL,
  status VARCHAR(30) NOT NULL,
  draft VARCHAR(255) NOT NULL,
  image_url VARCHAR(500) DEFAULT NULL,
  PRIMARY KEY (id),
  INDEX IDX_PROPERTY_AGENT (agent_id),
  CONSTRAINT FK_PROPERTY_AGENT
    FOREIGN KEY (agent_id) REFERENCES agent (id)
);

CREATE TABLE availability_slot (
  id INT AUTO_INCREMENT NOT NULL,
  agent_id INT DEFAULT NULL,
  start_at DATETIME NOT NULL,
  end_at DATETIME NOT NULL,
  is_available TINYINT NOT NULL,
  note VARCHAR(255) DEFAULT NULL,
  PRIMARY KEY (id),
  CONSTRAINT FK_SLOT_AGENT
    FOREIGN KEY (agent_id) REFERENCES agent (id)
);

CREATE TABLE appointment (
  id INT AUTO_INCREMENT NOT NULL,
  agent_id INT NOT NULL,
  slot_id INT NOT NULL,
  client_name VARCHAR(120) NOT NULL,
  client_email VARCHAR(180) NOT NULL,
  client_phone VARCHAR(30) DEFAULT NULL,
  message LONGTEXT DEFAULT NULL,
```

```
  created_at DATETIME NOT NULL,
  PRIMARY KEY (id),
  CONSTRAINT FK_APPOINTMENT_AGENT
    FOREIGN KEY (agent_id) REFERENCES agent (id),
  CONSTRAINT FK_APPOINTMENT_SLOT
    FOREIGN KEY (slot_id) REFERENCES availability_slot (id)
);

CREATE TABLE chat_message (
  id INT AUTO_INCREMENT NOT NULL,
  session_id VARCHAR(64) NOT NULL,
  role VARCHAR(20) NOT NULL,
  content LONGTEXT NOT NULL,
  client_name VARCHAR(120) DEFAULT NULL,
  client_email VARCHAR(180) DEFAULT NULL,
  created_at DATETIME NOT NULL,
  PRIMARY KEY (id),
  INDEX IDX_CHAT_SESSION (session_id)
);
```

Commande qui applique ce schéma :

```
php bin/console doctrine:migrations:migrate --no-interaction
```

VII. Conception de l'application (UML)

VII.a Diagrammes de cas d'utilisation

Le diagramme de cas d'utilisation place les fonctionnalités dans le système Assist'IA et les acteurs à l'extérieur. Le stéréotype « include » indique qu'un cas réutilise toujours un autre : prendre rendez-vous inclut la consultation des biens, et gérer ses annonces inclut la connexion. Ollama et l'API Adresse sont des acteurs secondaires, stéréotypés « service ».

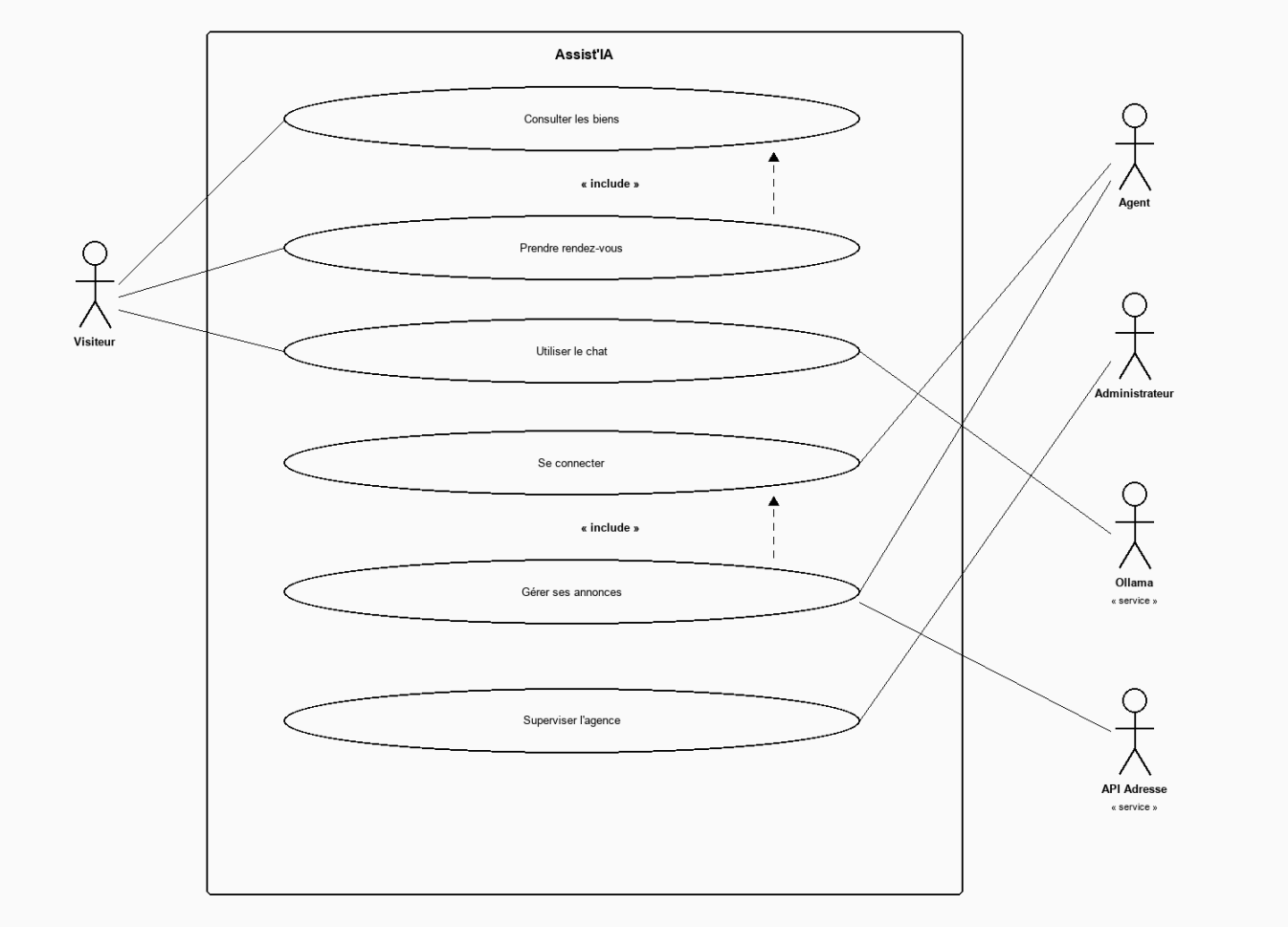


Figure VII.I : Diagramme de cas d'utilisation

Visiteur : consulter les biens, prendre rendez-vous, utiliser le chat, envoyer un message de contact.

Agent : se connecter, gérer ses annonces, consulter l'historique des conversations, gérer son calendrier.

Administrateur : superviser l'agence via EasyAdmin, ce qui suppose d'être connecté.

Système : générer les créneaux (commande console) et envoyer les e-mails de confirmation de rendez-vous.

VII.b Diagrammes de séquences

Deux scénarios sont dessinés : la connexion de l'agent, puis la prise de rendez-vous. Le chat est décrit ensuite, car il ajoute le repli sur les règles métier lorsque Ollama ne répond pas.

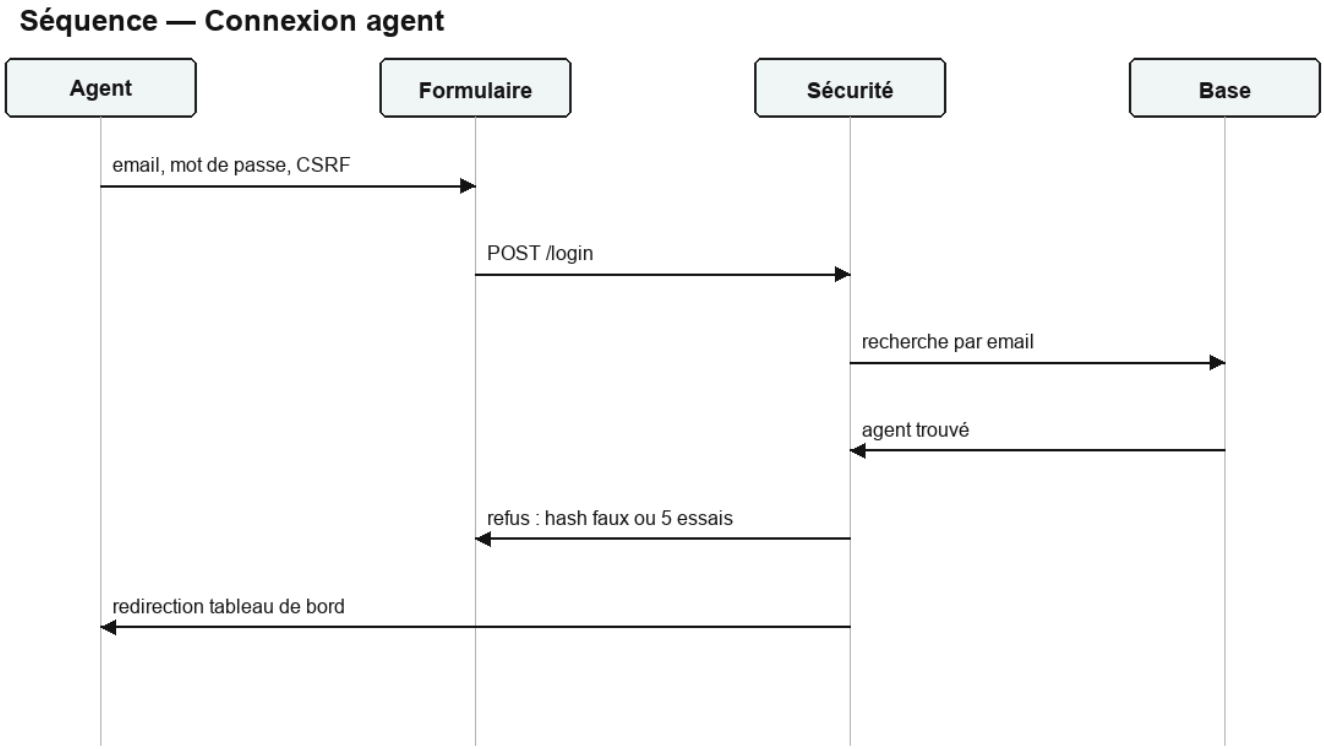


Figure VII.II : Séquence de connexion agent

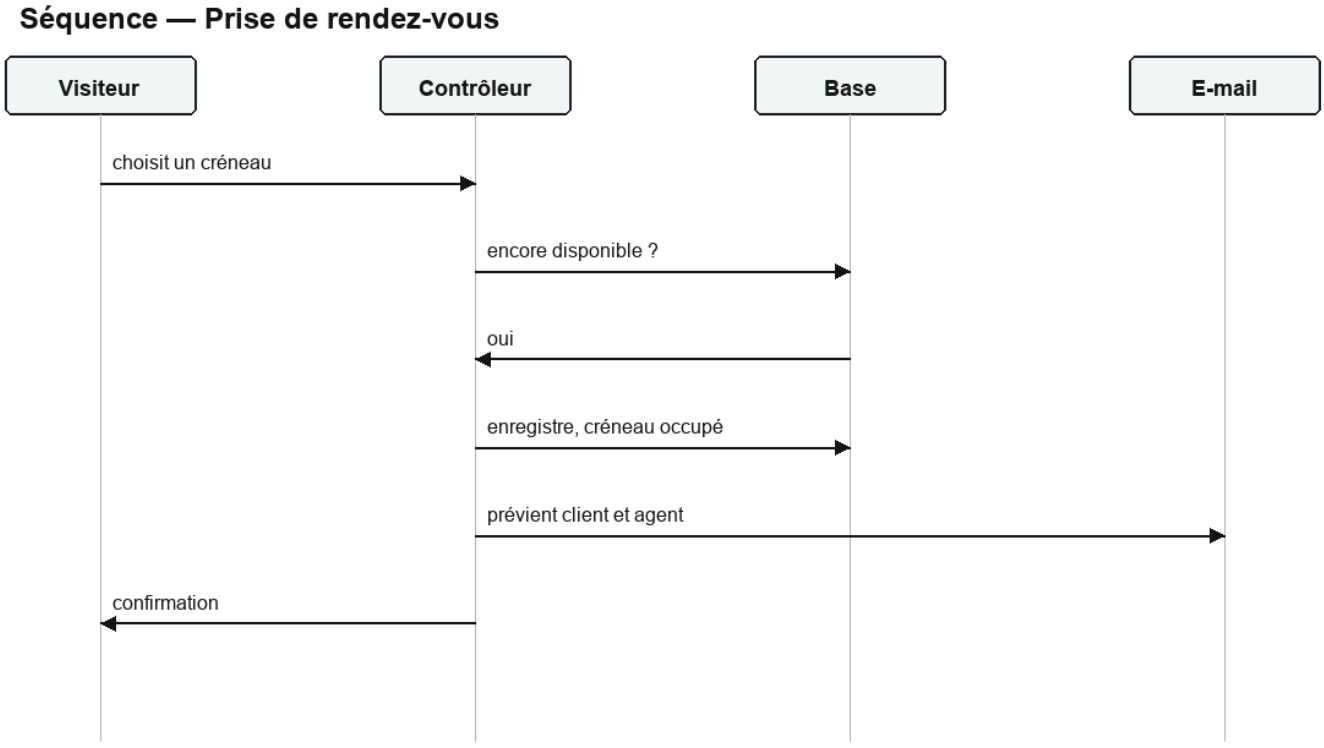


Figure VII.III : Séquence de prise de rendez-vous

Chat IA. Le visiteur saisit un message dans le composant React. Celui-ci appelle POST /api/chat. ChatController demande à ChatStorageService d'enregistrer le message utilisateur, puis à PropertyChatService de construire la réponse. Si Ollama est disponible,

OllamaChatService est appelé ; sinon, une réponse par règles (budget, typologie) est produite. Le message de l'assistant est enregistré, et le contrôleur renvoie un JSON contenant la réponse et l'identifiant de session.

Prise de rendez-vous. Le visiteur ouvre GET/POST /rdv. AppointmentController affiche le formulaire AppointmentType. À la soumission, le créneau est contrôlé (disponible, et rattaché à un agent). L'appointment est persisté et le créneau passe à isAvailable = false. AppointmentMailerService envoie un e-mail au client et à l'agent.

Authentification agent. L'agent poste le formulaire de /login (e-mail, mot de passe, jeton CSRF). Le pare-feu form_login de Symfony vérifie les identifiants avec le hasher, applique le throttling (5 tentatives / 15 minutes) et redirige vers le tableau de bord.

VII.c Diagrammes de classes UML

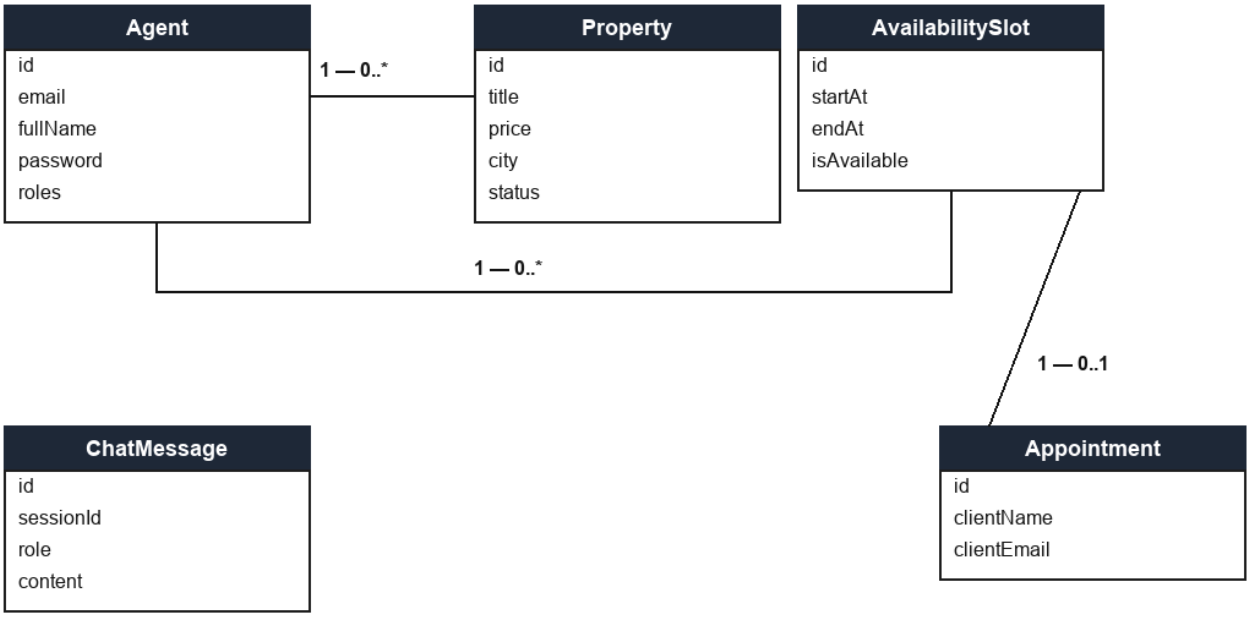


Figure VII.IV : Diagramme de classes

Entités principales :

- Agent — id, email, fullName, password, roles. Un agent possède plusieurs biens.
- Property — id, title, price, city, status, surface, rooms. Plusieurs biens pour un agent.
- AvailabilitySlot — id, startAt, endAt, isAvailable. Plusieurs créneaux pour un agent.
- Appointment — id, clientName, clientEmail, clientPhone. Un rendez-vous pour un agent et un créneau.
- ChatMessage — id, sessionId, role, content. Pas de clé étrangère : regroupement par sessionId.

Services métier : PropertyChatService (réponses du chat), OllamaChatService (client HTTP Ollama), ChatStorageService (persistance), AddressApiService (API data.gouv.fr), AvailabilitySlotGeneratorService (génération des créneaux), AppointmentMailerService (e-mails), PasswordResetService (jetons de réinitialisation).

Les services sont injectés dans les contrôleurs par le conteneur Symfony, ce qui sépare le traitement HTTP de la règle métier.

VIII. Architecture multi-couches

VIII.a Modèle MVC (couche logique)

Symfony implémente le patron Modèle-Vue-Contrôleur, qui sépare les responsabilités dans le code. Ce découpage est logique : il ne dit pas encore sur quelle machine chaque partie tourne.

- **Modèle.** Les entités Doctrine (Agent, Property, Appointment, AvailabilitySlot, ChatMessage) et les repositories, dans src/Entity/ et src/Repository/. Ils portent les données et l'accès à la base.
- **Vue.** Les templates Twig dans templates/, et le composant React du chat dans assets/app.js. Ils affichent, sans règle métier.
- **Contrôleur.** Les contrôleurs de src/Controller/ reçoivent la requête HTTP, appellent un service ou un repository, et renvoient du HTML ou du JSON.

L'exemple suivant est la création d'un bien, extraite de PropertyController. Le contrôleur ne contient pas la règle d'affichage : il délègue le formulaire au type Symfony, la donnée à l'entité, et l'écran au template.

```
// Contrôleur : reçoit la requête HTTP
public function new(Request $request, EntityManagerInterface $em): Response
{
    $property = new Property();                 // Modèle
    $form = $this->createForm(PropertyType::class, $property);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $user = $this->getUser();
        if ($user instanceof Agent) {
            $property->setAgent($user);
        }
        $em->persist($property);                // écriture en base
        $em->flush();
        return $this->redirectToRoute('app_property_index');
    }

    // Vue : le template Twig affiche le formulaire
    return $this->render('property/new.html.twig', [
        'property' => $property,
        'form' => $form,
    ]);
}
```

Lecture de l'exemple : l'agent appelle /property/new. Le contrôleur instancie le modèle Property, laisse PropertyType valider la saisie, enregistre via Doctrine, puis demande à la vue property/new.html.twig d'afficher le formulaire. Les trois couches restent distinctes dans le même fichier d'entrée.

VIII.b Architecture n-tiers (tiers physique)

L'architecture n-tiers décrit la répartition physique des composants, pas l'organisation du code. Là où le MVC sépare modèle, vue et contrôleur dans le projet Symfony, le n-tiers sépare les machines ou les processus : ce que voit le navigateur, ce qui exécute PHP, ce qui stocke les données, et les services appelés à l'extérieur. Assist'IA est déployé ainsi avec Docker.

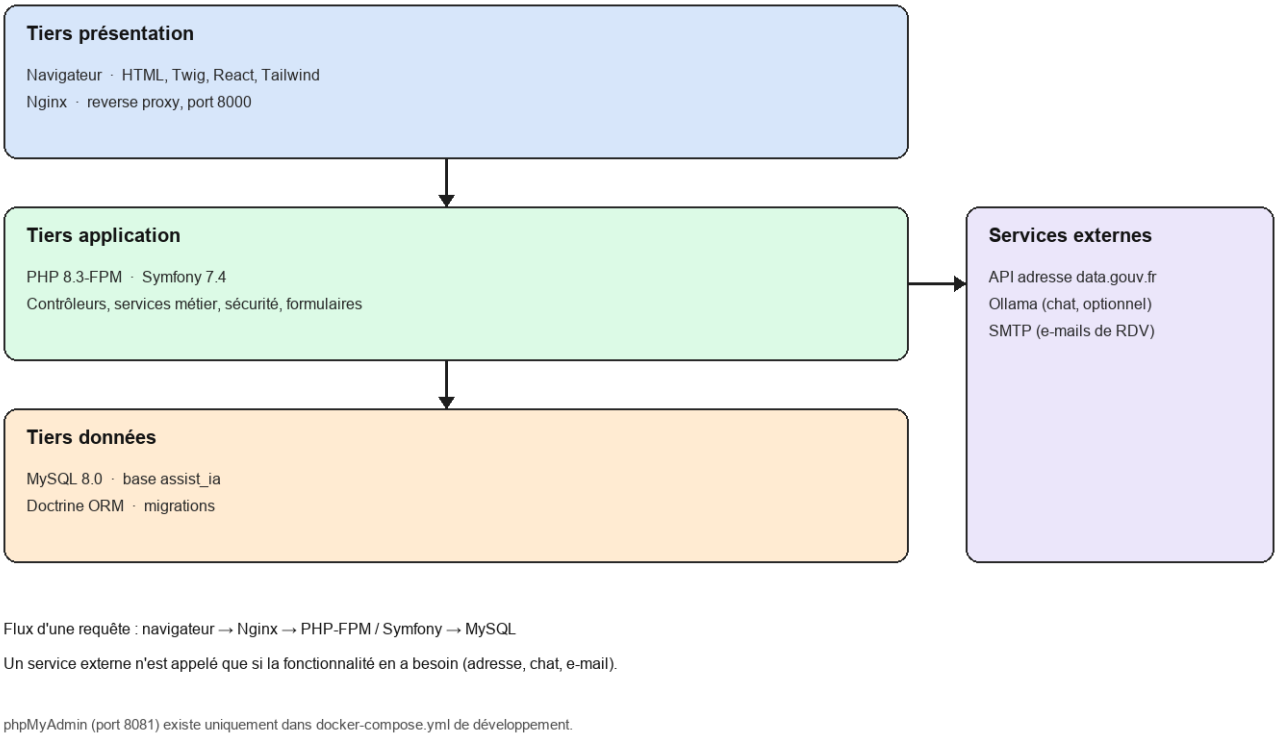


Figure VIII.I : Schéma de l'architecture n-tiers

Tiers présentation. Le navigateur affiche HTML, CSS, JavaScript et le widget React. Nginx sert de reverse proxy et écoute le port 8000.

Tiers application. PHP 8.3-FPM exécute Symfony 7.4 : contrôleurs, formulaires, services métier, sécurité.

Tiers données. MySQL 8.0 héberge la base assist_ia. Doctrine ORM est le seul accès aux données.

Services externes. L'API adresse data.gouv.fr, Ollama (chat, optionnel) et le SMTP des e-mails de rendez-vous. Ils ne sont appelés que si la fonctionnalité en a besoin.

Flux d'une requête : navigateur, puis Nginx (port 8000), puis PHP-FPM, puis Symfony, puis MySQL. En développement, phpMyAdmin (port 8081) permet d'administrer la base. Il est absent du fichier docker-compose.prod.yml.

VIII.c Séparation des responsabilités

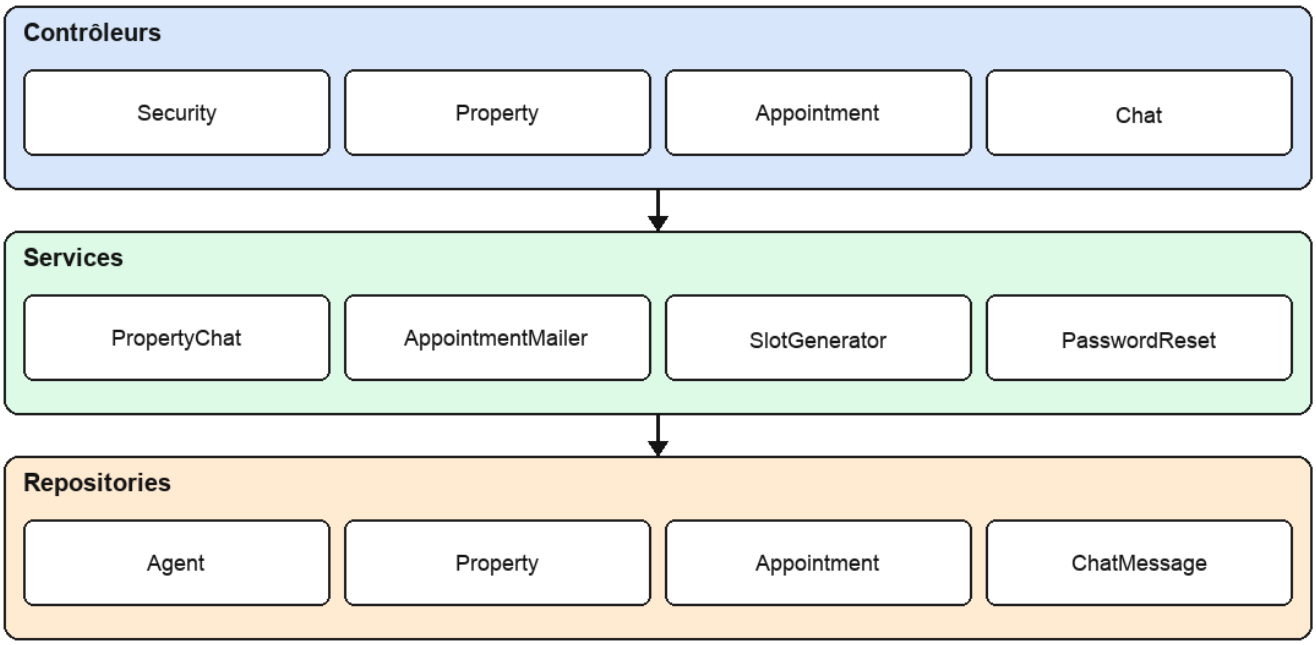


Figure VIII.II : Organisation contrôleurs, services, repositories

Couche	Dossier	Exemples
Contrôleurs	src/Controller	PropertyController, AppointmentController, ChatController
Services	src/Service	PropertyChatService, AppointmentMailerService
Entités	src/Entity	Agent, Property, Appointment, ChatMessage
Repositories	src/Repository	PropertyRepository, AppointmentRepository
Vues	templates/	Twig, et le chat React dans assets/app.js

Méthode	Route	Accès
GET	/	Public
GET	/biens	Public
GET, POST	/rdv	Public
GET, POST	/contact	Public
POST	/api/chat	Public
POST	/login	Public, avec jeton CSRF
GET	/agent, /property, /agent/calendrier	Agent
GET	/api/address/search	Agent
GET	/admin	Administrateur

- Contrôleurs fins : la règle métier est dans les services (PropertyChatService, AppointmentMailerService).
- Repositories pour les requêtes complexes.
- Formulaires Symfony pour la validation et le mapping.
- Commandes console pour les tâches en lot (EnsureAvailabilitySlotsCommand, LoadDemoDataCommand).
- Pas de logique métier dans les templates Twig.

IX. Sécurité

La sécurité est une exigence du référentiel CDA. Chaque menace est définie avant les mesures prises dans Assist'IA. La configuration vit dans config/packages/security.yaml et dans les templates.

IX.a Injection XSS (Cross-Site Scripting)

Une injection XSS (Cross-Site Scripting) consiste à faire exécuter du code, en général du JavaScript, dans le navigateur d'un autre utilisateur. L'attaquant place ce code dans une donnée que le site réaffiche : un message de chat, un titre d'annonce, un nom. Le script tourne alors avec l'origine du site et peut lire la session ou modifier la page.

Dans Assist'IA, le chat et les formulaires réaffichent des textes saisis par les visiteurs. Sans échappement, un message pourrait contenir une balise script.

Mesures :

- Twig échappe automatiquement les variables affichées. Le filtre |raw n'est pas utilisé sur un contenu utilisateur.
- Le composant React du chat affiche les messages comme du texte (m.content), sans innerHTML.
- Les entrées sont validées par le Validator Symfony sur les entités et les formulaires.

IX.b Injection SQL

Une injection SQL consiste à modifier une requête en glissant du SQL dans une saisie. L'exemple classique est une apostrophe suivie de OR 1=1. Selon la requête visée, l'attaquant peut lire, modifier ou détruire des données, ou contourner une authentification.

Dans Assist'IA, la connexion, la recherche de biens et les filtres du chat s'appuient sur MySQL. Une concaténation de la saisie dans le SQL ouvrirait cette faille.

Mesures :

- Doctrine ORM envoie des requêtes paramétrées (DQL, QueryBuilder). La valeur utilisateur est un paramètre, pas un morceau de la requête.

- Aucune concaténation SQL brute d'une donnée saisie.
- Les repositories encapsulent les requêtes : les contrôleurs ne construisent pas de SQL.

IX.c Failles CSRF (Cross-Site Request Forgery)

Une attaque CSRF (Cross-Site Request Forgery) consiste à forcer un utilisateur authentifié à exécuter une action à son insu (par exemple supprimer un bien ou changer un mot de passe) depuis un site malveillant. Sans protection, le navigateur envoie automatiquement les cookies de session.

L'espace agent est authentifié. Sans CSRF, un attaquant pourrait tenter de déclencher une suppression de bien (/property/{id}) ou une action sensible via un formulaire piégé.

Mesures mises en place :

1. **Protection du login.** Dans security.yaml, enable_csrf: true sur form_login. Dans login.html.twig, un champ caché porte csrf_token('authenticate'). Si le jeton est absent ou invalide, la connexion est refusée.
2. **Protection des actions destructives.** Lors de la suppression d'un bien, le contrôleur vérifie isCsrfTokenValid('delete'.$id, …). Sans jeton valide, la suppression est bloquée.
3. **Configuration globale.** Le fichier config/packages/csrf.yaml active la protection CSRF des formulaires Symfony (submit, authenticate, logout).
4. **Formulaires Symfony.** Les formulaires PropertyType, AppointmentType, ContactType incluent automatiquement un jeton CSRF.

Résultat : toute requête POST sensible doit contenir un jeton unique lié à la session. Un site externe ne peut pas le connaître.

```
<input type="hidden" name="_csrf_token" value="csrf-token">
```

Figure IX.I : Jeton CSRF du formulaire de connexion

```
21            form_login:
22                login_path: app_login
23                check_path: app_login
24                default_target_path: agent_dashboard
25                enable_csrf: true
```

Figure IX.II : Activation CSRF sur le pare-feu de connexion

IX.d Attaques par force brute

Une attaque par force brute consiste à essayer un grand nombre de mots de passe, souvent automatiquement, jusqu'à trouver le bon. Sur un formulaire de connexion, l'attaquant rejoue la même requête en changeant le secret à chaque essai.

Dans Assist'IA, la cible est l'espace agent : un compte agent ouvre les annonces, les conversations et le calendrier.

Mesures :

- Login throttling Symfony : maximum 5 tentatives par 15 minutes (security.yaml, login_throttling).
- Messages d'erreur génériques. La réinitialisation du mot de passe ne révèle pas si l'e-mail existe.

```
18          login_throttling:
19              max_attempts: 5
20              interval: '15 minutes'
```

Figure IX.III : Limitation des tentatives de connexion

IX.e Hachage des mots de passe

Le hachage transforme un mot de passe en une empreinte. L'opération est à sens unique : on ne retrouve pas le mot de passe à partir de l'empreinte. Un algorithme adapté (bcrypt ou Argon2) est volontairement lent et sale chaque empreinte avec une valeur aléatoire, le sel. Si la base fuite, les mots de passe en clair ne sont pas lisibles, et deux utilisateurs ayant le même mot de passe n'ont pas la même empreinte.

Mesures :

- password_hashers: auto. Symfony choisit bcrypt ou Argon2 selon la version de PHP.
- Les mots de passe ne sont jamais stockés en clair, ni écrits dans les journaux.
- La réinitialisation passe par un jeton aléatoire de 64 caractères hexadécimaux, valable une heure, et non par l'envoi de l'ancien mot de passe.

IX.f Conformité RGPD

Le RGPD (règlement général sur la protection des données) encadre, dans l'Union européenne, la collecte et le traitement des données personnelles. Il impose d'informer la personne, de ne collecter que ce qui est nécessaire, de sécuriser les données, et de permettre l'accès, la rectification et l'effacement.

Assist'IA traite des données personnelles : e-mail et nom de l'agent, nom, e-mail et téléphone du client qui réserve, contenu du chat.

Mesures :

- Page /politique-confidentialite (templates/legal/privacy.html.twig).
- Lien accessible depuis le footer de toutes les pages.

- Mention des droits d'accès, de rectification et d'effacement.
- Messages de chat stockés avec un sessionId, sans traceur tiers.
- Mots de passe hachés. Aucune revente de données.

IX.g Contrôle d'accès par rôles

Le contrôle d'accès vérifie, avant d'exécuter une action, que l'utilisateur possède le rôle requis. Un visiteur non connecté ne doit pas ouvrir l'espace agent, et un agent ne doit pas ouvrir la supervision réservée à l'administrateur.

Route	Rôle requis
/admin	ROLE_ADMIN
/agent, /property	ROLE_AGENT
/api/address/search	ROLE_AGENT
/api/chat	PUBLIC_ACCESS
Pages publiques	PUBLIC_ACCESS

Hiérarchie : ROLE_ADMIN hérite de ROLE_AGENT. Un administrateur peut donc tout ce que peut un agent, plus la supervision.

Lecture synthétique au regard de l'OWASP :

Risque OWASP	Mesure dans Assist'IA
A01 Contrôle d'accès	Rôles sur /admin, /agent et /property
A02 Secrets	Mots de passe hachés, secrets dans .env
A03 Injection	Doctrine paramétré, Twig et React qui échappent le texte
A05 Configuration	phpMyAdmin retiré en production, APP_DEBUG à 0
A07 Authentification	CSRF sur le login, 5 essais maximum en 15 minutes

X. Politique de tests

X.a Tests unitaires

Les tests unitaires ciblent les services métier isolément, sans requête HTTP.

Fichier	Objet testé
PropertyChatServiceTest.php	Extraction du budget, réponses par typologie
AddressApiServiceTest.php	Mapping de l'API adresse, requête vide

Fichier	Objet testé
AvailabilitySlotGeneratorServiceTest.php	Génération des créneaux
AppointmentMailerServiceTest.php	Envoi d'e-mail sans exception

X.b Tests fonctionnels

Les tests fonctionnels utilisent WebTestCase et une base SQLite recréée à chaque suite (DatabaseWebTestCase).

Fichier	Scénarios
SecurityControllerTest.php	Login agent, zone /property protégée
ChatControllerTest.php	Budget manquant, filtrage des biens
PublicPropertyControllerTest.php	Catalogue publié seulement, brouillon en 404
AppointmentControllerTest.php	Page accessible, réservation d'un créneau
PasswordResetControllerTest.php	Mot de passe oublié, e-mail inconnu, jeton valide
AddressControllerTest.php	Authentification requise, requête manquante

X.c Résultats et intégration CI

Résultat actuel : 23 tests, 53 assertions. Exécution locale : php bin/phpunit. Les mêmes tests tournent dans le pipeline CI à chaque push sur main.

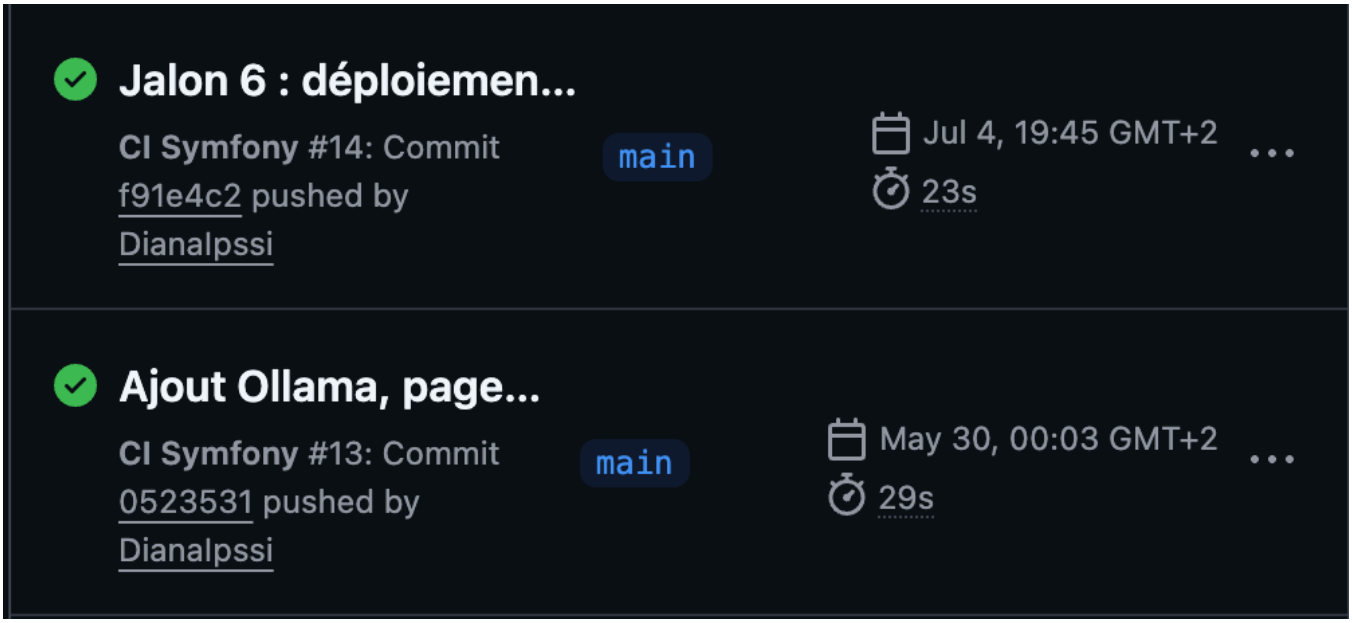


Figure X.I : Pipeline CI exécutant les tests PHPUnit

X.d Jeu d'essai représentatif

Test fonctionnel — réservation. Entrée : créneau disponible, nom et e-mail valides. Attendu : appointment créé, créneau isAvailable = false. Obtenu : conforme (AppointmentControllerTest.php).

Test unitaire — budget du chat. Entrée : message « budget 150000 ». Attendu : budget extrait = 150000. Obtenu : conforme (PropertyChatServiceTest.php).

XI. Déploiement et mise en production

XI.a Conteneurisation avec Docker

Docker empaquete l'application et ses dépendances dans une image, de sorte que le poste de développement, la CI et la production exécutent le même PHP. L'image est décrite par le Dockerfile. La capture suivante est le script du projet.

Dockerfile

```
FROM php:8.3-fpm

RUN apt-get update && apt-get install -y \
    git unzip libicu-dev zlib1g-dev g++ libxml2-dev libzip-dev \
    && docker-php-ext-install intl pdo pdo_mysql zip \
    && rm -rf /var/lib/apt/lists/*

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html

COPY docker/php/conf.d/app.ini /usr/local/etc/php/conf.d/app.ini
COPY docker/php/entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
CMD ["php-fpm"]
```

Figure XI.1 : Dockerfile de l'application

Lecture du script. FROM php:8.3-fpm part de l'image officielle PHP. RUN installe git, unzip et les bibliothèques, puis compile les extensions intl, pdo_mysql et zip. COPY récupère le binaire Composer. WORKDIR fixe le dossier de l'application. Les deux COPY suivants déposent la configuration PHP et le script d'entrée. ENTRYPOINT lance ce script ; CMD démarre php-fpm.

En développement, docker-compose.yml ajoute MySQL 8.0 (port hôte 3307), Nginx (port 8000) et phpMyAdmin (port 8081). L'entrypoint installe les dépendances, joue les migrations, charge les données de démonstration et génère les créneaux.

```
docker compose up --build
docker compose exec php php bin/console doctrine:migrations:migrate --no-
interaction
docker compose exec php php bin/console app:load-demo-data
```

L'application répond alors sur http://localhost:8000. Extrait allégé de la composition des services :

```
services:
  database:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: assist_ia
  php:
    build:
      dockerfile: Dockerfile
    environment:
      APP_ENV: prod
      APP_DEBUG: "0"
  nginx:
    image: nginx:1.27-alpine
    ports:
      - "8000:80"
```

XI.b Environnements de déploiement

Environnement	Fichier	APP_ENV	phpMyAdmin
Développement	docker-compose.yml	dev	Oui
Production	docker-compose.prod.yml	prod	Non

En production, les secrets passent par .env.local : APP_SECRET, MYSQL_ROOT_PASSWORD, MAILER_DSN. APP_DEBUG vaut 0.

XI.c Stratégies de mise en production

- Image Docker validée par le CD (tag v1.0).
- Démarrage : docker compose -f docker-compose.prod.yml up --build -d.
- Migrations jouées une fois, pas à chaque redémarrage.
- Politique de redémarrage unless-stopped sur les conteneurs de production.
- Ollama peut être coupé (OLLAMA_ENABLED=false) : le chat retombe sur les règles métier.

```
git tag -a v1.0 -m "Version finale"
git push origin v1.0
```

XII. Veille technologique

XII.a Veille sur les technologies du projet

Technologie	Source de veille
Symfony 7	symfony.com/blog, SymfonyCasts
PHP 8.3	php.net/releases, PHPWatch
Doctrine ORM	doctrine-project.org
Docker	docs.docker.com, Docker Blog
Tailwind CSS	tailwindcss.com/blog
OWASP	owasp.org — sécurité web
ANSSI	ssi.gouv.fr — guides sécurité
Ollama / LLM	ollama.com, notes de version GitHub

Exemple concret : Symfony 7.4 et l'Asset Mapper ont guidé le choix de React via importmap.php plutôt qu'un bundler Webpack lourd.

XII.b Veille globale du secteur

- PropTech : digitalisation des agences immobilières, chatbots.
- RGPD : évolutions de la CNIL sur les cookies et le consentement.
- DevOps : généralisation de la CI/CD et des conteneurs en production.
- Accessibilité : RGAA, prise en compte du handicap.

XIII. Veille sécurité

XIII.a Analyse des menaces actuelles

Menace	Risque pour Assist'IA	Criticité
XSS	Contenu du chat saisi par l'utilisateur	Moyenne — traitée
CSRF	Formulaires de login et de suppression	Haute — traitée
Force brute	Espace agent	Haute — throttling
Injection SQL	Requêtes en base	Haute — ORM
Fuite de données	E-mails, conversations	Moyenne — RGPD

XIII.b Captures et exemples concrets

Les captures de throttling et de jeton CSRF sont commentées dans la partie sécurité. Le pipeline CI, qui rejoue les tests à chaque push, fait partie du filet de sécurité : une régression sur l'authentification ou la réservation casse la branche avant qu'elle ne parte en production.

XIII.c Mesures préventives

- Mise à jour régulière des dépendances (composer update, audit dans la CI).
- Pas de secret en dur dans le code : .env et secrets GitHub.
- phpMyAdmin absent en production.
- APP_DEBUG=0 en production.
- Validation stricte des entrées de formulaires.

XIV. Difficultés rencontrées

Plusieurs difficultés techniques et d'organisation ont jalonné Assist'IA. Elles sont détaillées avec la solution retenue. Ces obstacles m'ont appris à isoler un problème, à le reproduire, puis à le fermer par un test.

XIV.a Problèmes techniques et solutions

1. Intégration d'Ollama. Ollama n'est pas toujours disponible, et la latence varie (timeout de 90 s). Solution : double stratégie dans PropertyChatService. Appel d'Ollama s'il répond, sinon réponses par règles métier. L'URL Docker est host.docker.internal:11434.

2. Parsing des critères de recherche. L'IA renvoie parfois un JSON mal formé. Solution : extraction par expression régulière et json_decode avec gestion d'erreur dans resolveSearchCriteria().

3. Recherche par typologie (T2, studio). Le chatbot ne filtrait pas correctement les biens par type. Solution : amélioration de extractSearchKeyword() et de la requête du repository.

4. Concurrence sur les créneaux. Risque théorique de double réservation simultanée. Solution : vérification de isAvailable avant l'enregistrement. Piste d'amélioration : verrou pessimiste Doctrine.

5. Entrypoint Docker en développement. Les migrations et les données de démonstration partent à chaque démarrage du conteneur. C'est acceptable en développement. En production, les migrations sont jouées une fois, comme le décrit le README.

6. Formulaire de contact sans envoi d'e-mail. En développement, le contact affiche un message de succès sans SMTP réel. La limitation est documentée. MAILER_DSN est configurable pour la production.

XIV.b Obstacles organisationnels

- Tenir le tableau Kanban les semaines où l'alternance prend le pas sur la formation.
- Choisir, lorsqu'une carte grossit, de la découper plutôt que de dépasser la date de revue.

XIV.c Apprentissages tirés

- Prévoir un repli quand un service externe (Ollama) est instable.
- Les tests automatisés évitent de recasser une carte déjà dans Terminé.
- Docker aligne le poste, la CI et la production.
- Documenter au fil de l'eau, pas la veille de la soutenance.

XV. Expérience en alternance — Innbox / DataLake

XV.1 Contexte entreprise

Innbox est une entreprise de self storage (location de box) avec plusieurs centres en région PACA (Antibes, Mandelieu, Nice, Saint-Isidore). L'équipe IT est composée de deux personnes : mon manager et moi. Durant l'alternance, j'ai occupé le poste de Junior Data Engineer. J'étais responsable du projet DataLake, de la conception à la mise en production.

XV.2 Problématique et objectifs

Innbox utilisait plusieurs applications métier isolées : Kinnovis (réservations, factures), Aircall (appels), Airtable (base interne), Axiome (QR codes et accès, sans API). L'objectif était de centraliser les données dans un DataLake et de visualiser des indicateurs en temps réel via Grafana, pour les équipes commerciales et la direction.

XV.3 Solution technique

Pipeline : les API Kinnovis, Aircall et Airtable sont lues par des scripts Python, déclenchés par un CRON GitHub Actions (all.py). Les CSV sont déposés sur AWS S3, catalogués par Glue, interrogés par Athena, puis affichés dans Grafana. Les identifiants sont des secrets GitHub, jamais écrits dans le code. Les données Axiome, sans API, sont extraites manuellement en attendant une ouverture.

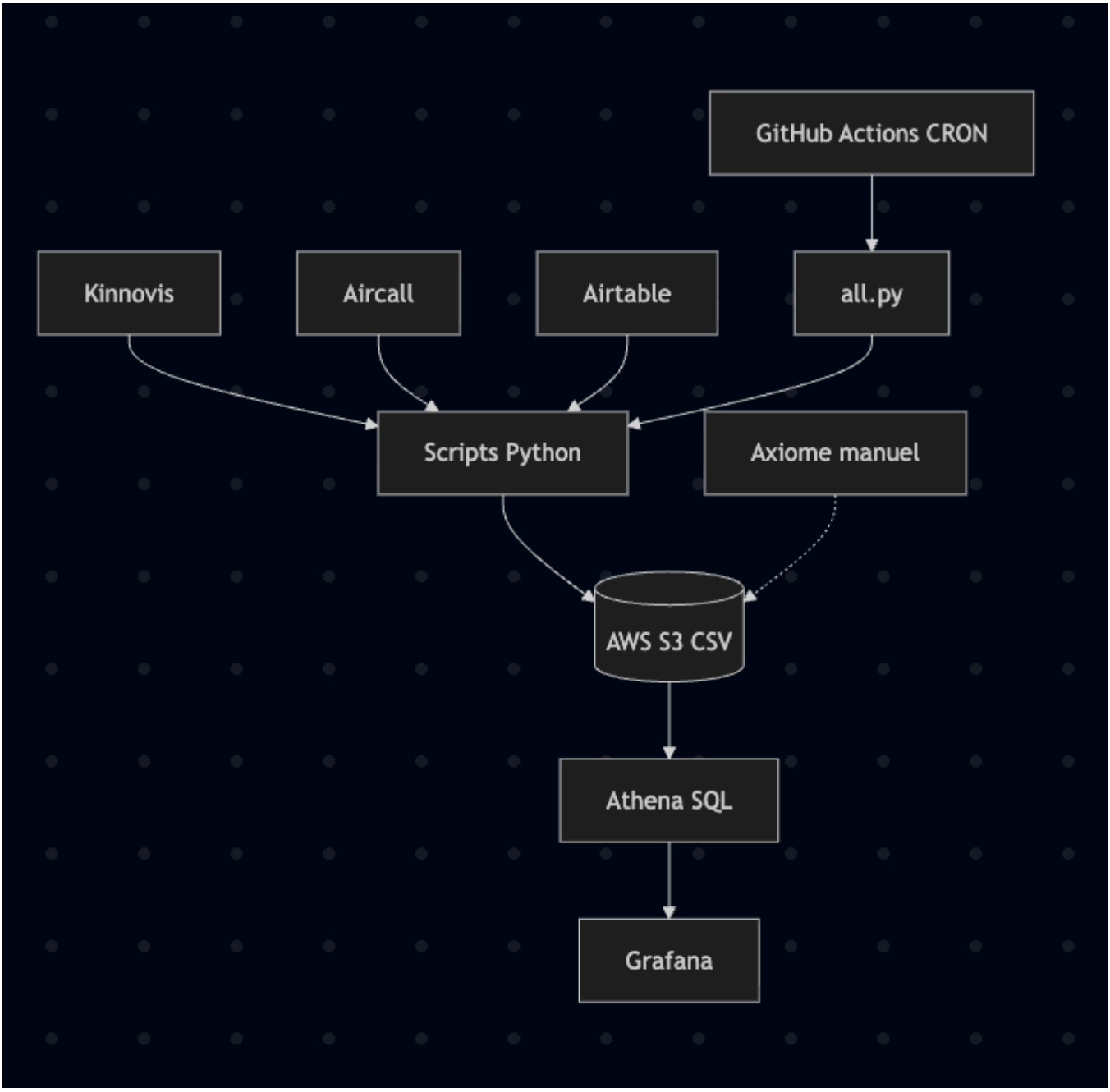


Figure XV.I : Architecture du DataLake Innbox

XV.4 Réalisations principales

- Mise en place du DataLake : architecture, ingestion, transformation.
- Normalisation des CSV pour Athena (dates, téléphones, nommage, dédoublonnage).
- Tableaux Grafana : taux de décrochage Aircall, move-in par centre, taux d'occupation, remplissage par catégorie de box.
- Automatisation GitHub Actions : plus besoin de lancer les scripts à la main.
- Migration de Grafana d'un poste local vers un NUC dédié, avec le prestataire réseau.

XV.5 Difficultés et lien avec Assist'IA

Difficulté	Solution
Compatibilité CSV / Athena	Formatage strict des dates et des téléphones
Évolutions métier (agent IA Aircall, plans Kinnovis)	Adaptation des scripts et des requêtes Grafana
Grafana dépendant de mon PC	Déploiement sur un NUC
Pas d'API Axiome	Extraction manuelle locale

Lien avec Assist'IA : les deux projets utilisent GitHub Actions pour l'automatisation (CI/CD côté Symfony, CRON côté data). La logique est la même, les piles techniques diffèrent.

XV.6 Impact métier

- Gain de temps : les pipelines tournent sans intervention.
- Indicateurs et rendez-vous accessibles aux commerciaux et au directeur.
- Projet opérationnel ; un affichage mural est prévu.

XVI. Conclusion et ouverture

XVI.a Bilan du projet

Assist'IA est une application web fonctionnelle qui répond au cahier des charges et couvre les compétences du titre Concepteur Développeur d'Applications :

- Conception : Merise, UML, architecture MVC et n-tiers
- Développement : interfaces Twig et React, services métier, API REST
- Sécurité : authentification, CSRF, throttling, hachage, RGPD
- Qualité : 23 tests PHPUnit, CI automatisée
- Déploiement : Docker en développement et en production, CD sur tags

Le tableau Kanban a rendu l'avancement visible chaque jour. L'expérience Innbox complète ce parcours : un projet data mené en autonomie, avec les mêmes réflexes d'automatisation.

XVI.b Perspectives d'évolution

- Envoi réel de l'e-mail de contact et notification de l'agent.
- Verrou pessimiste sur la réservation des créneaux.
- Publication de l'image Docker vers un registry.
- Tests de bout en bout (Panther ou Cypress).

- Audit d'accessibilité RGAA.

XVI.c Compétences acquises

Compétence	Mise en œuvre
Environnement de travail	Docker, Git, Symfony CLI
Interfaces utilisateur	Twig, Tailwind, React
Composants métier	Services PHP, API REST
Base relationnelle	Doctrine, migrations, Merise
Gestion de projet	Kanban : Backlog, À faire, En cours, Terminé
Sécurité	OWASP, ANSSI, RGPD
Tests	PHPUnit unitaire et fonctionnel
DevOps	GitHub Actions, Docker de production
Data (alternance)	Python, AWS, Grafana, CRON GitHub Actions

XVII. Remerciements

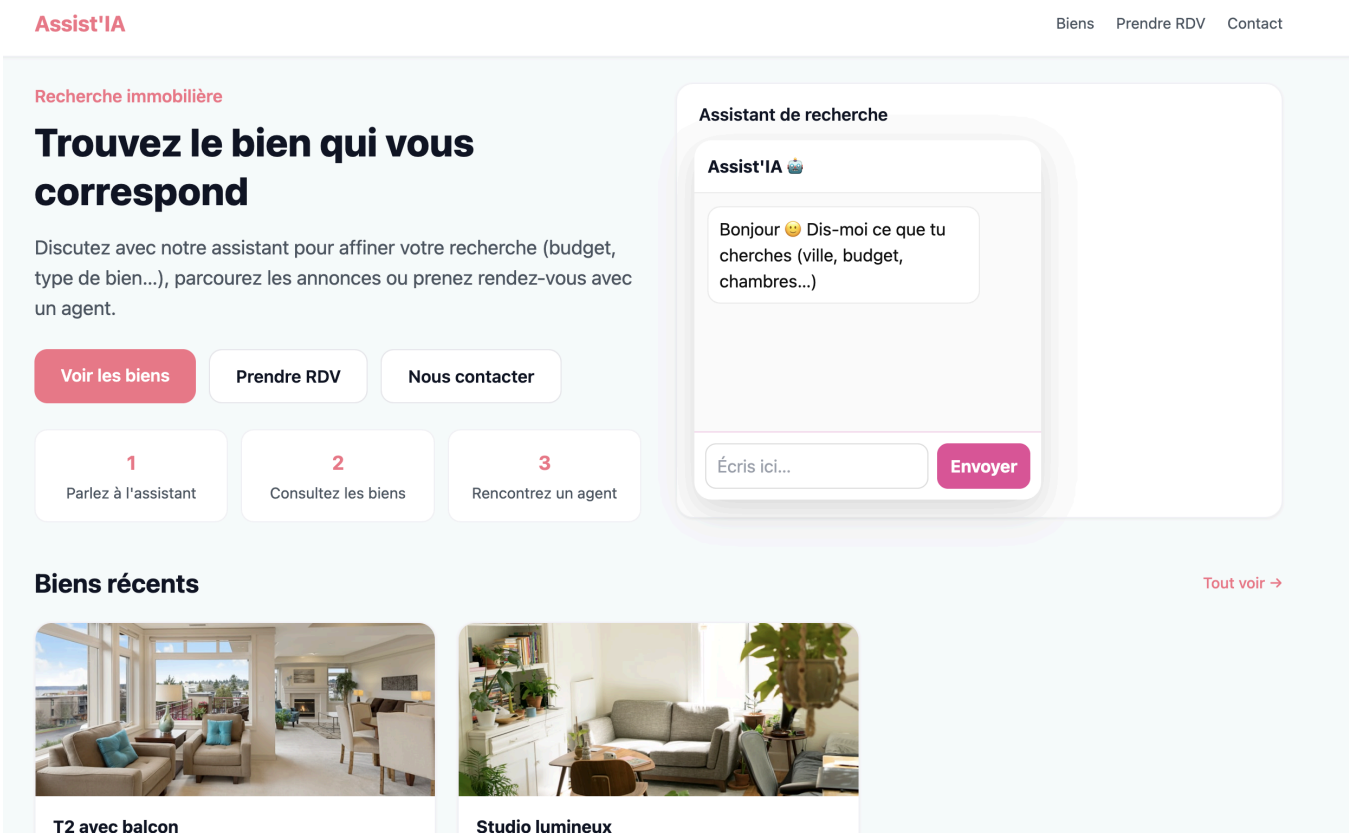
Je remercie :

- Hugo LIEGEARD, formateur IPSSI, pour l'accompagnement, la checklist et les conseils de soutenance ;
- Carole et l'équipe IPSSI pour la formation ;
- mon manager et le président d'Innbox pour la confiance accordée sur le DataLake.

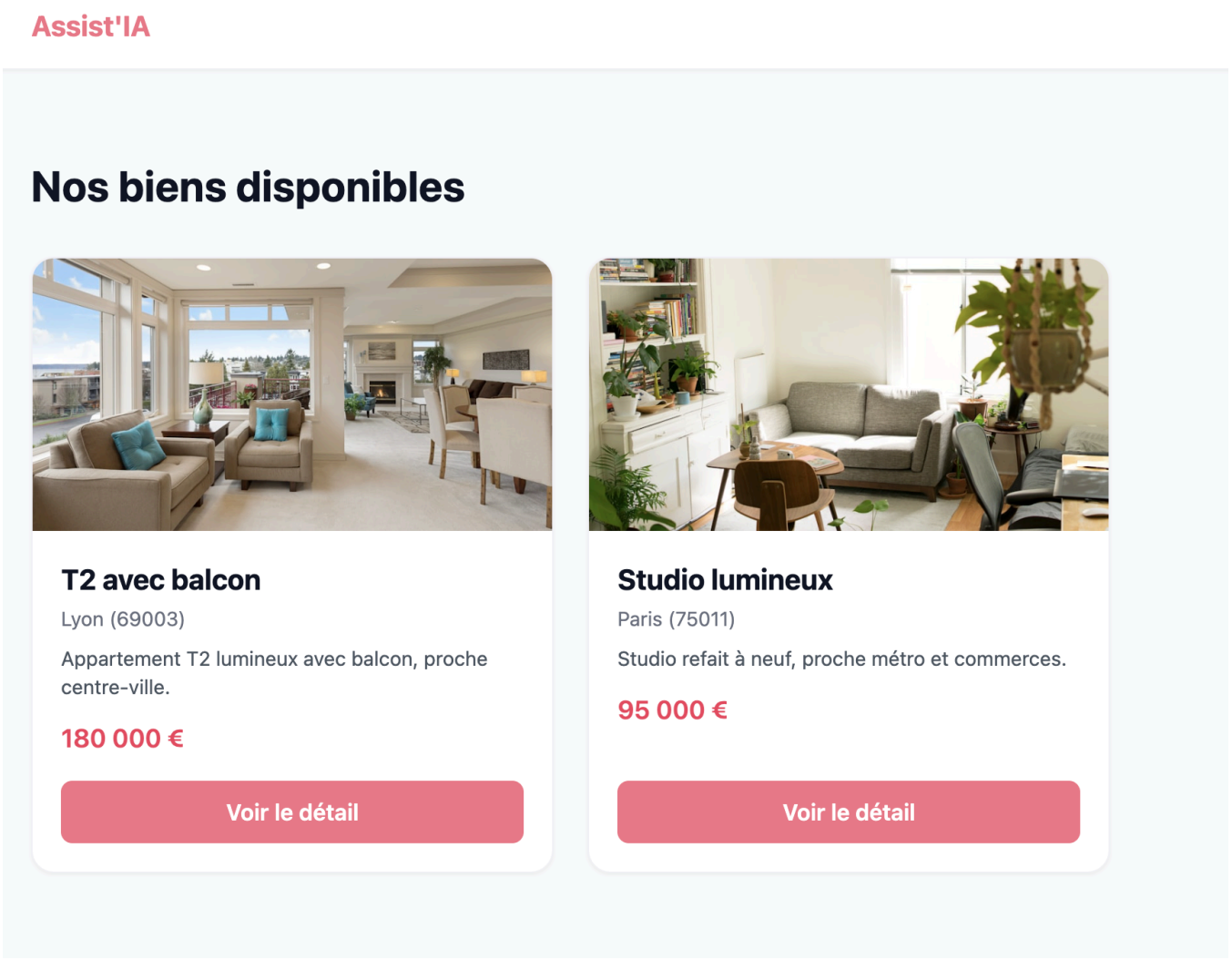
XVIII. Annexes

Les annexes regroupent les captures qui illustrent le fonctionnement de l'application, les tests, les workflows, la page RGPD, Docker et le projet Innbox. La liste paginée figure en tête de dossier.

Annexe A1 — Captures de l'application



Écran de l'application Assist'IA



Écran de l'application Assist'IA

Annexe A2 — Résultats des tests PHPUnit

```
diana@mbp-de-dl assist-ia % php bin/phpunit
PHPUnit 11.5.50 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.5.2
Configuration: /Users/diana/Desktop/assist-ia/phpunit.dist.xml

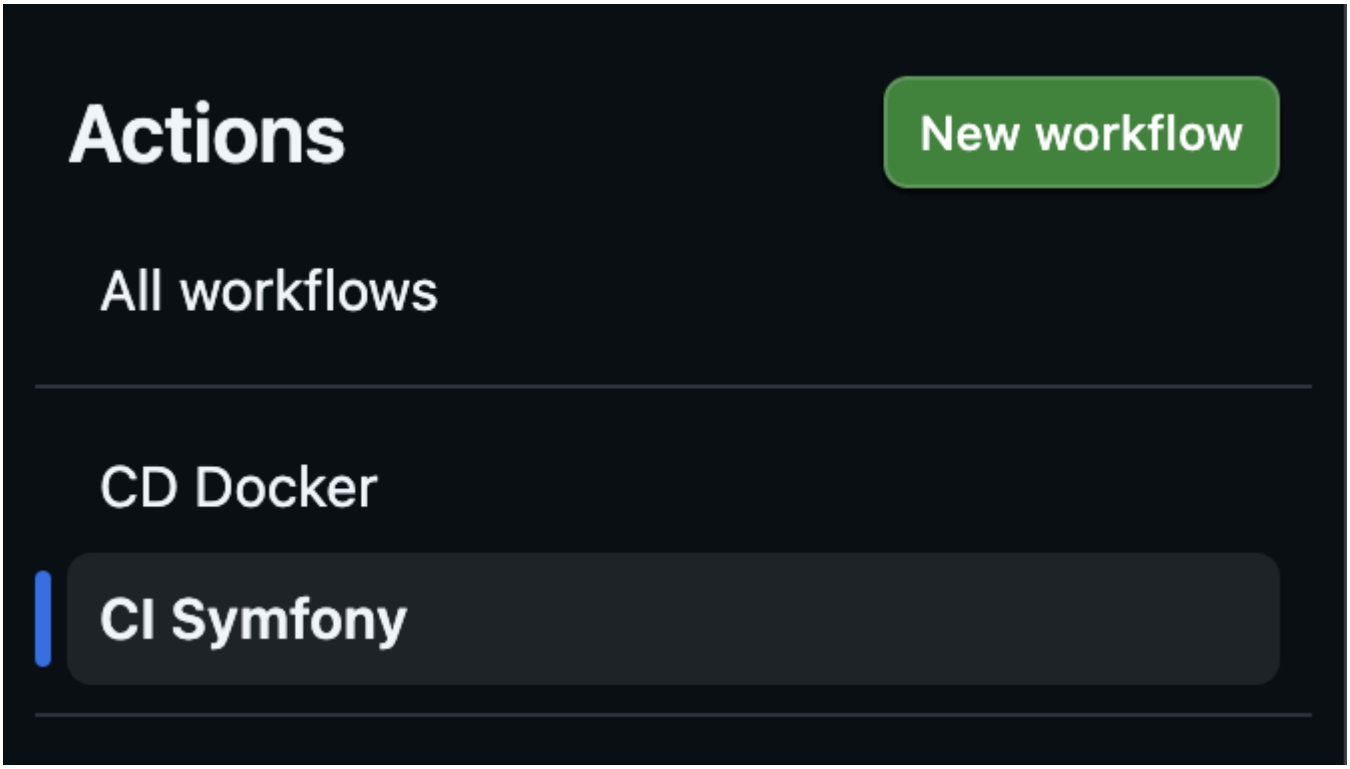
.......................                                           23 / 23 (100%)

Time: 00:00.843, Memory: 54.50 MB

OK (23 tests, 53 assertions)
diana@mbp-de-dl assist-ia %
```

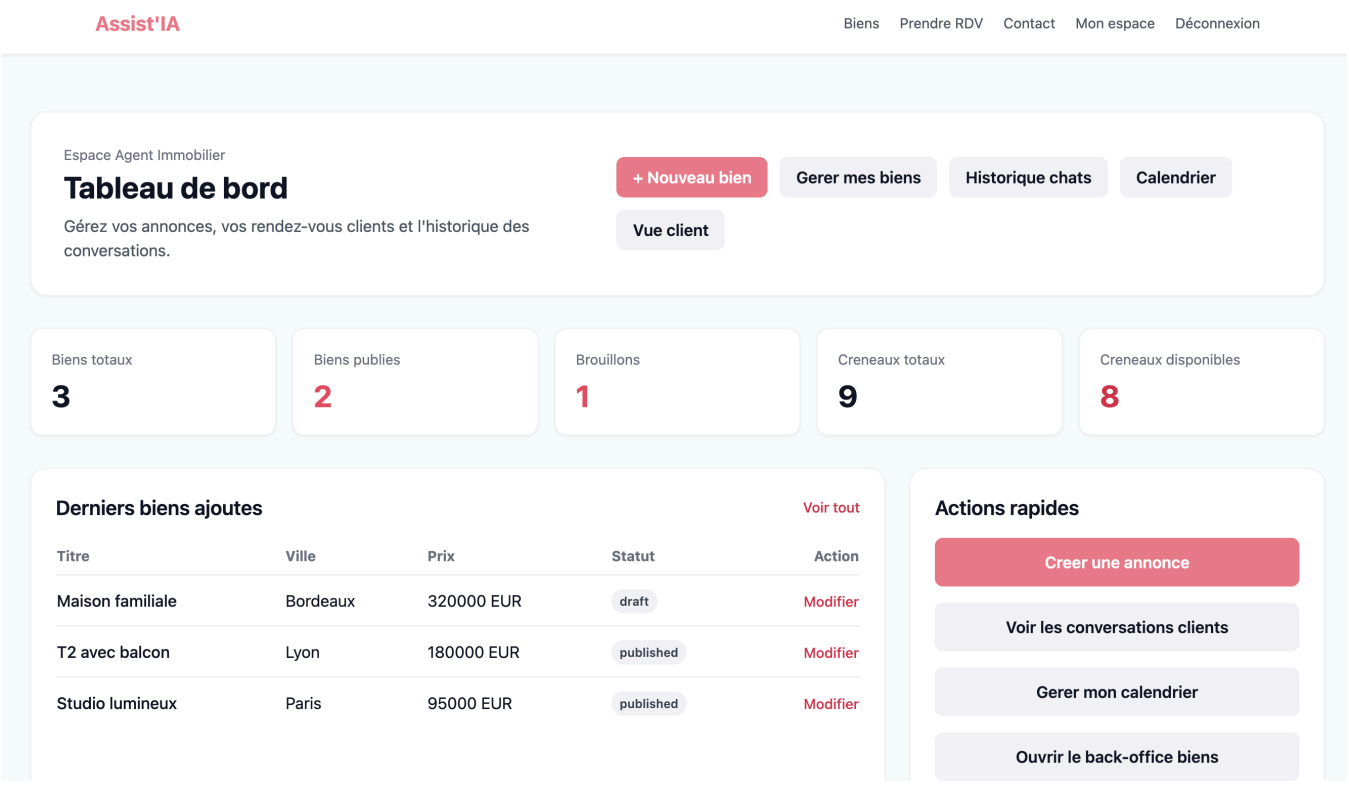
Sortie des tests automatisés

Annexe A3 — Workflows GitHub Actions

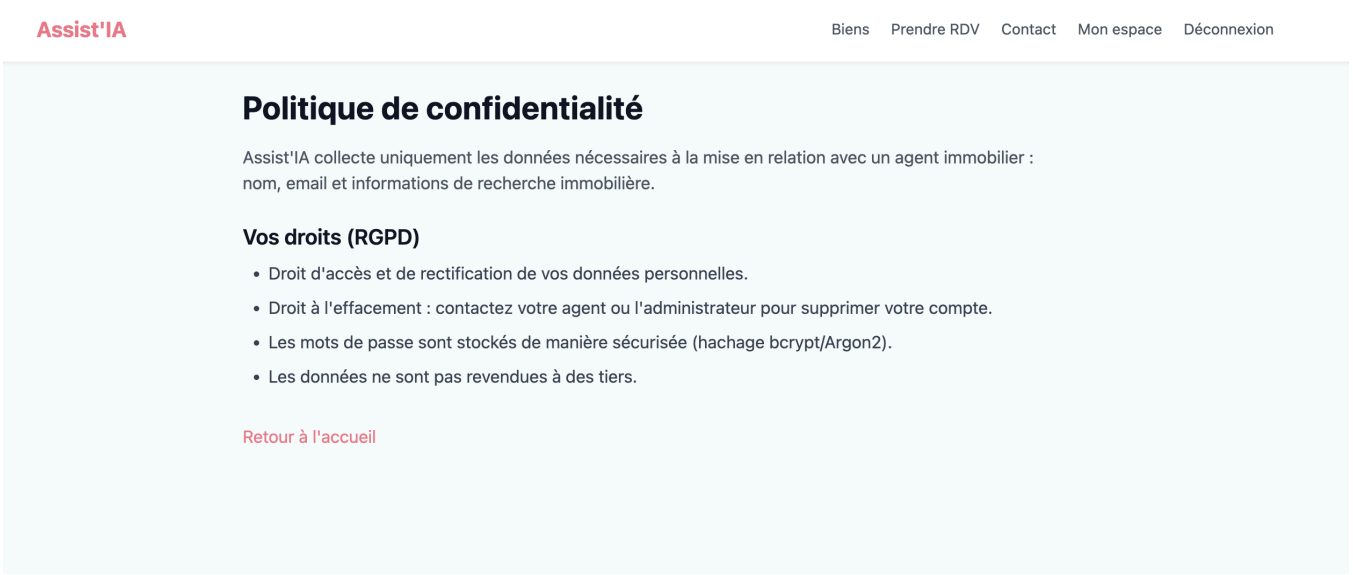


Workflows CI Symfony et CD Docker

Annexe A4 — Page de politique de confidentialité



Page RGPD



Page RGPD, suite

Annexe A5 — Fichiers Docker Compose

docker-compose.yml

```yaml
services:
  database:
    image: mysql:8.0
    container_name: assist_db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: assist_ia
    ports:
      - "3307:3306"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-proot"]
      interval: 5s
      timeout: 5s
      retries: 10

  php:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: assist_php
    volumes:
      - .:/var/www/html
    environment:
      APP_ENV: dev
      APP_SECRET: ChangeMeInProduction
      DATABASE_URL: mysql://root:root@database:3306/assist_ia?serverVersion=8.0&charset=utf8mb4
      OLLAMA_ENABLED: "true"
      OLLAMA_BASE_URL: "http://host.docker.internal:11434"
      OLLAMA_MODEL: "llama3.2"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    depends_on:
      database:
        condition: service_healthy

  nginx:
    image: nginx:1.27-alpine
    container_name: assist_nginx
    ports:
      - "8000:80"
    volumes:
```

Composition des services

```yaml
    environment:
      APP_ENV: dev
      APP_SECRET: ChangeMeInProduction
      DATABASE_URL: mysql://root:root@database:3306/assist_ia?serverVersion=8.0&charset=utf8mb4
      OLLAMA_ENABLED: "true"
      OLLAMA_BASE_URL: "http://host.docker.internal:11434"
      OLLAMA_MODEL: "llama3.2"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    depends_on:
      database:
        condition: service_healthy

  nginx:
    image: nginx:1.27-alpine
    container_name: assist_nginx
    ports:
      - "8000:80"
    volumes:
      - .:/var/www/html:ro
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - php

  phpmyadmin:
    image: phpmyadmin:5
    container_name: assist_phpmyadmin
    ports:
      - "8081:80"
    environment:
      PMA_HOST: database
      PMA_USER: root
      PMA_PASSWORD: root
    depends_on:
      database:
        condition: service_healthy

volumes:
  db_data:
```

Environnement de développement

docker-compose.prod.yml

```yaml
services:
  database:
    image: mysql:8.0
    container_name: assist_db_prod
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: assist_ia
    volumes:
      - db_data_prod:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-p${MYSQL_ROOT_PASSWORD:-root}"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

  php:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: assist_php_prod
    volumes:
      - .:/var/www/html
    environment:
      APP_ENV: prod
      APP_DEBUG: "0"
      APP_SECRET: ${APP_SECRET:-ChangeMeInProduction}
      DATABASE_URL: mysql://root:${MYSQL_ROOT_PASSWORD:-root}@database:3306/assist_ia?serverVersion=8.0&charset=utf8mb4
      MAILER_DSN: ${MAILER_DSN:-null://null}
      OLLAMA_ENABLED: ${OLLAMA_ENABLED:-false}
      OLLAMA_BASE_URL: ${OLLAMA_BASE_URL:-http://host.docker.internal:11434}
      OLLAMA_MODEL: ${OLLAMA_MODEL:-llama3.2}
    extra_hosts:
      - "host.docker.internal:host-gateway"
    depends_on:
      database:
        condition: service_healthy
    restart: unless-stopped

  nginx:
    image: nginx:1.27-alpine
    container_name: assist_nginx_prod
    ports:
```

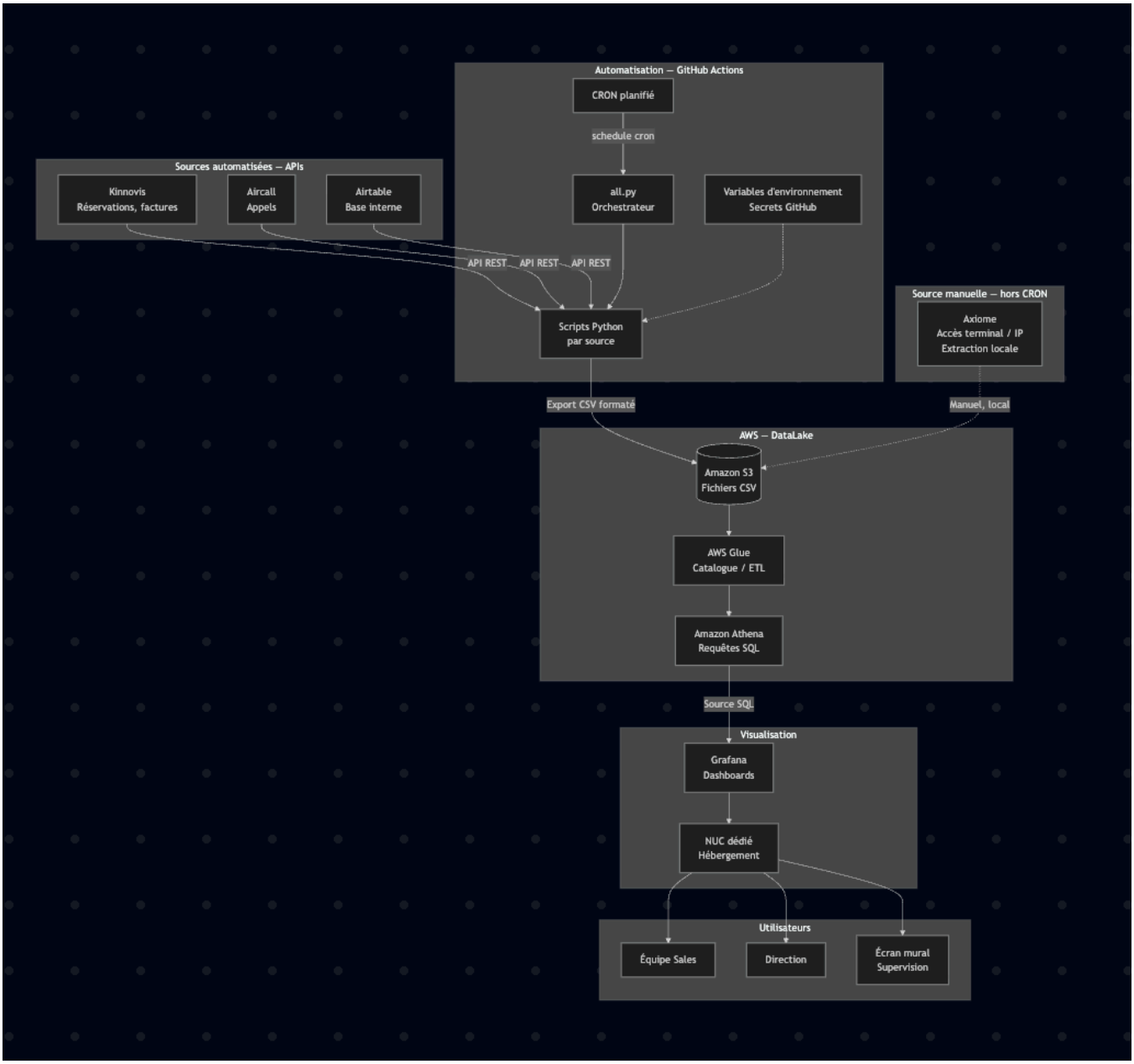
Environnement de production

```yaml
  nginx:
    image: nginx:1.27-alpine
    container_name: assist_nginx_prod
    ports:
      - "${APP_PORT:-8000}:80"
    volumes:
      - .:/var/www/html:ro
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - php
    restart: unless-stopped

volumes:
  db_data_prod:
```

Environnement de production, suite

Annexe A6 — Schéma d'architecture DataLake



Architecture DataLake Innbox

Annexe A7 — Tableaux de bord Grafana



Tableau Grafana



Tableau Grafana, suite